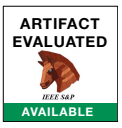

InsPIRe: Communication-Efficient PIR with Server-side Preprocessing

Rasoul Akhavan Mahdavi
Google and University of Waterloo
rasoul.akhavan.mahdavi@uwaterloo.ca

Sarvar Patel
Google
sarvar@google.com

Joon Young Seo
Google
jyseo@google.com

Kevin Yeo
Google and Columbia University
kwlyeo@google.com

***Abstract*—We present InsPIRe that is the first private information retrieval (PIR) construction simultaneously obtaining both high-throughput and low query communication while using only server-side preprocessing (meaning no offline communication). Prior PIR schemes with both high-throughput and low query communication required substantial offline communication of either downloading a database hint that is 10-100x larger than the communication cost of a single query (such as SimplePIR and DoublePIR [Henzinger *et al.*, USENIX Security 2023]) or streaming the entire database (such as Piano [Zhou *et al.*, S&P 2024]). In contrast, recent works such as YPIR [Menon and Wu, USENIX Security 2024] avoid offline communication at the cost of increasing the query size by 1.8-2x, up to 1-2 MB per query. Our new PIR protocol, InsPIRe, obtains the best of both worlds by obtaining high-throughput and low communication without requiring any offline communication. Compared to YPIR, InsPIRe requires 5x smaller cryptographic keys, requires up to 50% less online query communication while obtaining up to 25% higher throughput. We show that InsPIRe enables improvements across a wide range of applications and database shapes including the InterPlanetary File System and private device enrollment.**

At the core of InsPIRe, we develop a novel ring packing algorithm, InspiRING, for transforming LWE ciphertexts into RLWE ciphertexts. InspiRING is more amenable to the server-side preprocessing setting that allows moving the majority of the necessary operations to offline preprocessing. InspiRING only requires two key-switching matrices whereas prior approaches needed logarithmic key-switching matrices. We also show that InspiRING has smaller noise growth and faster packing times than prior works in the setting when the total key-switching material sizes must be small. To further reduce communication costs in the PIR protocol, InsPIRe performs the second level of PIR using homomorphic polynomial evaluation, which only requires one additional ciphertext from the client.

1. Introduction

Private information retrieval (PIR) is a powerful cryptographic protocol enabling users to privately query entries from a public database held by a server. In this protocol, the server holds a database with N entries and the client wishes to retrieve the i-th entry. PIR ensures the client's query index i remains hidden from the server. The strong privacy guarantees of PIR have been critically leveraged to design privacy-preserving systems for many applications such as advertising [38], blocklists [46], certificate transparency [42], contact discovery [9], [32], [45], databases [77], file systems [59], media consumption [39], metadata-hiding communication [65], [5], [48], [4], [1], password leak checks [76], [53], [3] and web search [41]. PIR has also seen adoption for real-world deployments in industry by Apple [6], Google [79] and Microsoft [50].

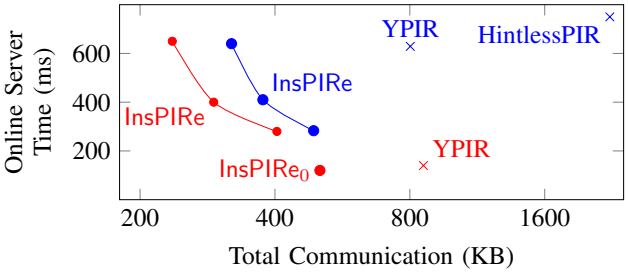

Figure 1: Communication Cost vs. Online Server Runtime for a PIR query over a 1 GB database. Red points retrieve 1-bit entry and blue points retrieve a 32KB entry.

PIR has been studied in both the single-server (such as [47], [70], [69]) and multiple, non-colluding server settings (including [23], [34], [11]). While multi-server PIR schemes are efficient, their privacy relies on stronger, non-collusion trust assumptions that may be difficult to materialize in practice. In our work, we focus exclusively on single-server PIR to avoid the these challenges.

When evaluating the efficiency of PIR protocols, there are two core cost metrics that must be considered. First, the amount of computational cost necessary to serve PIR requests. The computational cost can be measured using throughput, which is the ratio of the database size and the server time (equivalently, the amount of time needed per database entry). The second important metric is the communication (bandwidth) needed for each PIR query. Over the past decade, many works have studied the problem of designing efficient PIR protocols that we survey below.

Server-Stored Client-Specific Keys. Starting from XPIR [60], there has been a long line of work designing PIR protocols based on the RLWE hardness assumption

DOI 10.1109/SP63933.2026.00061

including [4], [3], [66], [1], [61], [16]. These works obtain small online communication, but have two main drawbacks. First, they require substantially larger computational costs during queries compared to other PIR schemes (see below). Even worse, these works requires the server to store client-specific keys of several megabytes in size that must be used during PIR queries, which is very limiting in practice. These client-specific keys must be uploaded to the server (for example, either with the first query or in an offline preprocessing step). So, the total communication cost is substantially large, especially if users only perform a small number of PIR queries. Also, server storage grows with the number of users limiting applicability for systems with very large user bases.

Client-Stored Database Hints. Another line of work has considered the case where the PIR protocol with offline preprocessing where the client must retrieve and store hints about the database. Several recent works including SimplePIR and DoublePIR [42] as well as FrodoPIR [29] build LWE-based PIR schemes in this class. They obtain higher throughput compared to the prior RLWE-based PIR schemes while maintaining slightly larger, but comparable query communication. However, the database hint, that must be downloaded by each user before issuing any PIR queries, is substantially large. Furthermore, hints must be re-downloaded by all users each time the database changes.

Taking this paradigm to an even more extreme, recent work [26] has shown that client-stored database hints can obtain PIR schemes with online query time sublinear in the database size obtaining even higher throughput. Unfortunately, practical instantiations including [80], [75] require every user to stream the entire database during offline preprocessing. Furthermore, the database must be streamed every $\tilde{O}(\sqrt{n})$ PIR queries unlike the prior LWE-based schemes [42], [29]. The very recent work of ThorPIR [36] avoid streaming the entire database by, instead, using heavy cryptographic primitives requiring up to multiple hours of server computation for just one client to retrieve a hint.

Server-side Preprocessing with No Offline Communication. The main drawback of the prior two classes of PIR schemes usually come from the communication required in the offline preprocessing steps. To solve this, recent works including Tiptoe [41], HintlessPIR [52], YPIR [62], RLWEPIR [59], WhisPIR [31], and KSPIR [57] construct PIR protocols in the Common Reference String (CRS) model where there is no offline communication. This solves many of the problems from the prior schemes that require offline communication. Unfortunately, these works end up sacrificing online communication during PIR queries to eliminate offline communication. For example, YPIR [62] requires up to megabytes of online communication more compared to PIR with client-stored hints.

In a recent breakthrough work, Lin *et al.* [55] showed the first PIR scheme with silent preprocessing and sublinear query time. Unfortunately, the concrete cost remains large to date rendering the construction impractical [68].

Our Question. Given the current state of affairs in PIR, we are left to navigate one of two obstacles. If we insist on both high-throughput and low query communication, then we must solve the challenges and costs associated with the offline communication for either server-stored client-specific keys or client-downloaded database hints. In contrast, if we aim to avoid offline communication and require PIR schemes with silent preprocessing, we must stomach substantial additional communication for each PIR query. This leads to the following question:

Can we construct a PIR without offline communication obtaining high-throughput and low query communication?

In this work, we answer in the affirmative with a new class of PIR protocols that obtain both high-throughput and low query communication without offline communication.

1.1. Technical Overview and Contributions

Efficient PIR with Server-side Preprocessing. As our main contribution, we present a new PIR protocol, InsPIRe, that simultaneously obtains high-throughput and low query communication using only server-side preprocessing in the CRS model, without any offline communication. Specifically, we obtain higher throughput and lower communication than all state-of-the-art PIR protocols in the CRS and silent preprocessing model such as YPIR [62] and HintlessPIR [52]. We observe this advantage across various database sizes with different payload sizes. For example, for retrieving a 32 KB payload from a 32 GB database, InsPIRe has 10% higher throughput and 67-90% lower communication costs compared to HintlessPIR [52] and YPIR [62]. Moreover, we can parameterize InsPIRe to optimize for other metrics, e.g., lower communication. InsPIRe can achieve 78-93% reduction in communication whilst still having better throughput than HintlessPIR and YPIR. Figure 1 shows a full comparison with related work.

Ring Packing with Preprocessing. At the core of our techniques, we develop a novel ring packing algorithm InspiRING that translates LWE ciphertexts into a RLWE ciphertext (Section 3) using three steps. In the first step, LWE ciphertexts are transformed to an intermediate representation with the message encoded as a constant term of the plaintext polynomial. This is achieved by carefully applying the trace function, expressed in terms of the two generators of the Galois group. Next, ciphertexts are aggregated to obtain one ciphertext in the intermediate representation that encodes all the original messages. Finally, the result is converted back to the standard RLWE ciphertext format using only two cryptographic keys. In the context of PIR, InspiRING requires substantially less cryptographic material to compress PIR responses while simultaneously obtaining better throughput.

Homomorphic Polynomial Evaluation. We also present a novel technique to further reduce the size of the PIR response by representing database entries using polynomials. We show how to reduce the PIR response size by evaluating polynomials homomorphically in an efficient manner (Section 4).

Applications. We show that our new protocol, InsPIRe, can be applied to improve efficiency in several real-world deployments. To showcase the improvements across a wide-range of realistic database sizes and shapes, we show that InsPIRe can improve the efficiency of two applications that heavily rely upon PIR with silent preprocessing. First, we show that InsPIRe is a better alternative for issuing private queries in the InterPlanetary File System (IPFS), where PIR queries are issued over databases of various sizes with varying payload sizes. In this context, our work can improve communication by up to 51% and computation by up to 86% over prior work [59]. Secondly, we consider the private device enrollment that is deployed by Google [79] where PIR is used to privately check membership of identifiers in a database. We show InsPIRe requires less than 300 KB communication to respond to a request in about 800 ms, for a database of 40M identifiers. In summary, InsPIRe enables improvements across a wide set of applications and databases.

2. Preliminaries

Basic Notation. We use lowercase bold letters to denote vectors and uppercase bold letters to denote matrices. We use $[a_1, \ldots, a_n]$ to denote an $1 \times n$ row matrix and $[a_1, \ldots, a_n]^\top$ to denote an $n \times 1$ column matrix. For a set R, we use $R^n = R^{n\times 1}$ to denote the set of $n \times 1$ column vectors over R while $R^{1\times n}$ to denote the set of $1 \times n$ row vectors over R. We will use R and $R^{1\times 1}$ interchangeably. For a vector $\mathbf{a}$, we use $\mathbf{a}[j]$ to index the j-th element and $\mathbf{a}[i:j]$ to denote a slice in interval $[i, j)$. If it is clear from the context, we will treat a row vector as a tuple and vice versa and interchange them freely without explicit casting.

For a discrete probability distribution D defined over the set S, we use $x \leftarrow D(S)$ to denote sampling an element from S according to the distribution D. We use $U(S)$ to denote a uniform distribution of a set S.

We use λ to denote the security parameter.

Gaussian and Subgaussian Variables. A random variable X is subgaussian with parameter σ if $\Pr[|X| > t] \leq 2\exp(-\pi t^2/\sigma^2)$ for all $t \geq 0$. A discrete Gaussian variable with width σ is subgaussian with parameter σ. If X is subgaussian with σ, cX is subgaussian with parameter $|c|\sigma$. The sum $X_1 + \ldots + X_k$ of independent subgaussian variables (with parameters $\sigma_1, \ldots, \sigma_k$) is subgaussian with parameter $\sqrt{\sum_i \sigma_i^2}$. See [73] for more details.

Cyclotomic Rings. We use cyclotomic rings $R = \mathbb{Z}[X]/(X^d + 1)$ and $R_q = \mathbb{Z}_q[X]/(X^d + 1)$, where d is a power of two. Ring elements $p(X) \in R$ may be denoted as p if clear from context.

LWE, RLWE, and MLWE. In our work, we will use both LWE- and RLWE-based private key encryption. We use $\chi(\mathbb{Z})$ to denote an error distribution over $\mathbb{Z}$ and $\chi(R)$ to denote an error distribution over the coefficients of R. We naturally extend to $\chi(\mathbb{Z}^n)$ for an error distribution over $\mathbb{Z}^n$ and $\chi(R^n)$ for an error distribution over R^n.

An LWE encryption of a message $m \in \mathbb{Z}_p$ is the pair $(\mathbf{a}, b) \in \mathbb{Z}_q^d \times \mathbb{Z}_q$ such that $b = -\langle \mathbf{a}, \mathbf{s}\rangle + e + \Delta \cdot m$ where $\mathbf{s} \leftarrow \chi(\mathbb{Z}^d)$ is a secret key, $e \leftarrow \chi(\mathbb{Z})$ is a small error value, and Δ is a scaling factor that is typically $\Delta = \lfloor q/p \rfloor$. Note, the secret key $\mathbf{s}$ enables decryption message m by computing $b + \langle \mathbf{a}, \mathbf{s}\rangle = e + \Delta \cdot m \in \mathbb{Z}_q$. When e is small, this allows retrieving $m \in \mathbb{Z}_p$ by rounding.

An RLWE [35] encryption of a message $m \in R_p$ consists of the pair $(a, b) \in R_q \times R_q$ such that $b = -as + e + \Delta \cdot m$ where $s \leftarrow \chi(R)$ is a secret key, $e \leftarrow \chi(R)$ is an error polynomial and Δ is a scaling factor that is typically $\Delta = \lfloor q/p \rfloor$. Given the secret key s, it is possible to decrypt the message m by computing $b + as = e + \Delta \cdot m \in R_q$ such that m can be retrieved by rounding as long as e is small.

While we don't directly use it in our work, we draw some useful analogies to MLWE encryption schemes [13], [49] in our packing algorithm in Section 3. We refer to Section 2.3 in [13] for details.

Throughout the paper, we use d to denote both the dimension of LWE samples and the degree of RLWE samples. In both encryption schemes, we will use the version where the public random components of the ciphertexts are fixed, which remain secure as long as the secret keys are re-sampled (see [74]).

Gadget Matrices. [63] Given a modulus $q \in \mathbb{N}$ and a decomposition base $z \in \mathbb{N}$, we use $\mathbf{g_z} = [1, z, \ldots, z^{\ell-1}]^\top \in \mathbb{Z}_q^\ell$ where $\ell = \lceil \log q / \log z \rceil$ to denote a one-dimensional gadget matrix. We use $\mathbf{g_z^{-1}} : \mathbb{Z}_q \to \mathbb{Z}^{1\times\ell}$ to denote the base-z digit decomposition operator on an integer in $\mathbb{Z}_q$, where each component of the output vector is an integer in $[-z/2, z/2)$. We extend $\mathbf{g_z^{-1}} : \mathbb{Z}_q \to \mathbb{Z}^{1\times\ell}$ to $\mathbf{g_z^{-1}} : R_q \to R^{1\times\ell}$ to operate over the ring R_q where the digit decomposition is applied to each coefficient of the element in R_q independently.

RLWE-RGSW External Product [22]. For $\mathbf{g_z} \in \mathbb{Z}_q^\ell$ and identity matrix $\mathbf{I}_2$, let $\mathbf{G}_{2,z} = \mathbf{I}_2 \otimes \mathbf{g}_z \in R_q^{2\ell\times 2}$. An RGSW encryption of message $m \in R_p$ is $[\mathbf{a}', \mathbf{b}'] \in R_q^{2\ell\times 2}$:

$$[\mathbf{a}', \mathbf{b}'] := [\mathbf{a}, -s\mathbf{a} + \mathbf{e}] + m \cdot \mathbf{G}_{2,z}$$

where $\mathbf{a} \leftarrow U(R_q^{2\ell})$ is the random component vector and $\mathbf{e} \leftarrow \chi(R_q^{2\ell})$ is the noise vector. For a RLWE ciphertext $\mathsf{RLWE}(m_0) = (a, b)$ and a RGSW ciphertext $\mathsf{RGSW}(m_1) = [\mathbf{a}', \mathbf{b}']$, the external product is defined as:

$$\boxdot : \mathsf{RLWE}(m_0) \times \mathsf{RGSW}(m_1) \to \mathsf{RLWE}(m_0 m_1)$$
$$(a'', b'') := [\mathbf{g_z^{-1}}(a), \mathbf{g_z^{-1}}(b)] \cdot [\mathbf{a}', \mathbf{b}'].$$

The result is a RLWE ciphertext of the product $m_0 m_1$.

RLWE Key-Switching. Given a RLWE ciphertext $(a, b) = (a, -as + m + e)$ encrypted under a secret key s, the goal of the key-switching procedure is to transform the ciphertext (a, b) to be encrypted under a different key s'. Let z and ℓ be the gadget parameters. The key-switching procedure consists of a pair of algorithms $\mathsf{KS} = (\mathsf{Setup}, \mathsf{Switch})$ defined as:

- $\mathsf{KS.Setup}(s, s')$: On input source secret $s \in R_q$, target secret $s' \in R_q$

1) Sample $\mathbf{w} \leftarrow U(R_q^\ell)$ and $\mathbf{e} \leftarrow \chi(R^\ell)$.
2) Compute $\mathbf{y} \leftarrow -s' \cdot \mathbf{w} + s \cdot \mathbf{g}_z + \mathbf{e}$
3) Output $\mathbf{K} = [\mathbf{w}, \mathbf{y}] \in R^{\ell \times 2}$ as the key-switching matrix.

- $\mathsf{KS.Switch}((a,b), \mathbf{K})$: On input a RLWE ciphertext $(a,b) = (a, -as + m + e)$, a key-switching matrix $\mathbf{K} \in R^{\ell \times 2}$ that switches from s to s'
  1) Output $(a', b') \leftarrow (0, b) + \mathbf{g}_\mathbf{z}^{-1}(a) \cdot \mathbf{K}$.

We refer to Section 2.4 in [18] for more details.

Galois Group. For the cyclotomic ring R that we consider in this work, the Galois group is a group of ring automorphisms, with each element:

$$\tau_g : R \to R$$
$$\tau_g(p) = p(X^g)$$

for $g \in \{1, 3, \ldots, 2d-1\}$. The Galois group is isomorphic to the additive group $\mathbb{Z}_{d/2} \times \mathbb{Z}_2$ and can be generated using two generators [33].

For a non-negative integer i, we use τ_g^i (resp. τ_g^{-i}) to denote the Galois group element corresponding to i applications of τ_g (resp. τ_g^{-1}).

For a matrix $\mathbf{M} \in R^{n \times m}$, we extend the use of τ_g and denote $\tau_g(\mathbf{M})$ to mean that τ_g is applied to each entry of $\mathbf{M}$. We extend this notion naturally to τ_g^i as well.

Independence Heuristic. Following prior works such as [4], [66], [61], [52], we assume the independence heuristic where the error terms of all intermediate computations are independent. As a result, we analyze the variance of the noise vector as opposed to the worst case magnitude. In particular, we leverage the fact that variance is additive for independent subgaussian variables to analyze noise.

PIR. We consider single-server PIR schemes with single roundtrip queries consisting of the tuple of four algorithms $(\mathsf{Setup}, \mathsf{Query}, \mathsf{Respond}, \mathsf{Extract})$:

- $\mathsf{Setup}(1^\lambda, \mathbf{D}) \to (\mathsf{pp}, \mathbf{D}')$: Executed by the server, this algorithm receives security parameter 1^λ and database $\mathbf{D}$, and outputs public parameters pp and encoded database $\mathbf{D}'$. Output pp is shared with the client.
- $\mathsf{Query}(\mathsf{pp}, \mathsf{idx}) \to (\mathsf{st}, \mathsf{qry})$: Executed by the client, this algorithm receives public parameters pp and the DB index idx, and outputs client state st and query qry.
- $\mathsf{Respond}(\mathsf{pp}, \mathbf{D}', \mathsf{qry}) \to \mathsf{resp}$: Executed by the server, this algorithm receives public parameters pp, encoded database $\mathbf{D}'$, and query qry, and outputs response resp.
- $\mathsf{Extract}(\mathsf{pp}, \mathsf{st}, \mathsf{resp}) \to \mathsf{entry}$: Executed by the client, this algorithm receives public parameters pp, client state st, and server's response resp and outputs the entry.

Our work follows standard security and correctness definitions from prior works (such as [42], [62]). We defer formal descriptions to Appendix A.1.

2.1. Common Reference String (CRS) Model

Our construction works in the common reference string model that assumes that all parties have access to a global string, generated by a trusted entity. Under this assumption, we fix the random components of the LWE/RLWE ciphertexts, and the scheme remains secure as long as fresh secret keys are generated for each query (with multiplicative factor security loss proportional to the number of queries). Prior work such as SimplePIR [42], HintlessPIR [52], and YPIR [62] also work in this model.

2.2. Convention of our Pseudocode Presentation

In the CRS model, the message-independent components of the ciphertexts are fixed before the online phase, enabling precomputation of protocol parts dependent solely on these message-independent components. Typically, the offline and the online phases of a protocol are described using separate sets of pseudocode. Instead, we combine both phases into a single, integrated pseudocode description.

To clearly distinguish these phases within a unified structure, operations performable during offline preprocessiong are highlighted. This convention requires a two-pass interpretation of our protocol:

1) **Offline Preprocessing Pass:** Only highlighted portions are executed and generated data is cached. Non-highlighted parts are disregarded.
2) **Online Execution Pass:** Non-highlighted portions are executed using cached values from highlighted portions.

We believe this integrated style offers a more intuitive grasp of the interplay between precomputed and online operations without having to jump back and forth between them.

3. Ring Packing with Preprocessing

In this section, we present InspiRING, our ring packing construction transforming LWEs into a RLWE ciphertext. InspiRING is specifically designed for the CRS model where the random components of ciphertexts are fixed and can be preprocessed. InspiRING will be the core building block of our low query communication and high-throughput PIR protocol. Prior LWE-based PIR schemes [41], [52] essentially used existing packing/bootstrapping algorithms in a black-box manner, which mainly target a more general setting where the random parts of the ciphertexts are not necessarily known ahead and can be preprocessed [1]. Our observation is that, if we design algorithms where offline preprocessing utilize fixed random components of the ciphertexts, then we can obtain a more efficient ring packing algorithm. In comparison to the CDKS ring packing [18] used in YPIR [62], our construction is practically more efficient and has better analytical and concrete noise growth while only requiring *two key-switching matrices*. In contrast, CDKS [18] requires $\lg d$ key-switching matrices, increasing PIR communication.

Notational Convention. For the presentation in this section, we adopt a notational convention that omits the message

1. For PIR, these schemes did take advantage of preprocessing the random components of the ciphertexts to improve the online computation, but only by "simulating" the underlying algorithm in the offline phase.

scaling factor Δ and the error term e and assume they are part of the plaintext message. For example, the LWE pseudorandom component $b = -\langle \mathbf{a}, \mathbf{s} \rangle + e + \Delta m$ will be written as $b = -\langle \mathbf{a}, \mathbf{s} \rangle + m'$ where $m' = e + \Delta m$. For this section, we emphasize that this notational convenience does not impact the correctness or security of the described algorithm. On the other hand, the noise introduced during the packing will be written and analyzed explicitly.

3.1. Revisiting CDKS [18] Packing

Before we describe our construction, we will first re-examine the CDKS packing algorithm [18] which is also used in YPIR [62]. While certain insights from the CDKS algorithm will be directly incorporated into our construction, our primary focus here is to understand why CDKS requires many key-switching matrices. In this discussion, d represents both the dimension of the initial LWE ciphertexts and the degree of the target ring.

The CDKS algorithm begins by interpreting the input LWE ciphertexts denoted as $(\mathbf{a}, b = -\langle \mathbf{a}, \mathbf{s} \rangle + m) \in \mathbb{Z}_q^d \times \mathbb{Z}_q$ as elements of $R_q \times R_q$ where $R_q = \mathbb{Z}_q[X]/(X^d + 1)$. This embedding process treats the LWE ciphertext as an RLWE ciphertext, $(\tilde{a}, \tilde{b}) \in R_q \times R_q$. In this interpretation, $\tilde{b}$ corresponds to the original b (treated as a constant term polynomial), and $\tilde{a}$ is $\sum_{i=0}^{d-1} \mathbf{a}[i] \cdot X^{-i}$. Consequently, $\tilde{b}$ can be formulated as:

$$\tilde{b} = -\tilde{a}\tilde{s} + \tilde{m} \pmod{q}. \tag{1}$$

In this equation, $\tilde{s} = \sum_{i=0}^{d-1} \mathbf{s}[i] \cdot X^i$ represents the LWE secret $\mathbf{s}$ as a polynomial. A key outcome of this transformation is that the constant term of $\tilde{m}$ holds the original message m. However, the other coefficients of $\tilde{m}$ are filled with arbitrary values due to the embedding. If these other coefficients were zero, packing multiple messages would be simple: multiplying the RLWE ciphertext by X^i cyclically shifts its coefficients by i positions, and thus, by applying suitiable X^i multiplications and summing the resulting ciphertexts, a single RLWE ciphertext could embed all the LWE plaintexts.

For ring packing, it's necessary to zero out specific coefficient slots. This creates space for the plaintext messages from other ciphertexts to be merged into a single, consolidated RLWE ciphertext.

CDKS addresses this challenge by incrementally merging the input ciphertexts through a recursive process. This packing procedure can be viewed as a complete binary tree with depth of $\lg(d)$. The leaves of this tree are the RLWE-embedded LWE ciphertexts (Eq. 1). At each internal node, two RLWE ciphertexts are combined into a single, valid RLWE ciphertext. This merging step employs an automorphism to zero out the necessary coefficient slots of the plaintext message. Specifically, these automorphisms preserve certain coefficients (for already packed messages) while negating others. When a ciphertext is added to its automorphic image, the negated coefficients cancel each other, therby freeing up slots for the merging process. A subesequent key-switching operation is then performed to eliminate the extraneous term from the automorphism.

A critical aspect of this method is that the specific automorphism applied depends on the number of messages already packed. Consequently, each level in the recursion tree utilizes a distinct automorphism. This necessitates a unique key-switching matrix at each level to remove the extraneous term generated by the automorphism. As a result, this packing strategy inherently requires $\lg(d)$ key-switching matrices, which results in large cryptographic material.

It is important to note that the CDKS algorithm was not initially developed with the CRS model in mind. In the CRS model, the random components of ciphertexts are known and can be preprocessed. This raises a crucial question: can we leverage the CRS model to devise a more efficient ring packing algorithm to reduce the substantial cryptographic overhead of the existing methods such as CDKS?

3.2. Packing with Two Key-switching Matrices

We affirm the above question by presenting our packing algorithm that can efficiently translate d input LWE ciphertexts into a single RLWE ciphertext using only *two key-switching matrices*. Here, d is the dimension of the input LWE ciphertexts and also the degree of the target ring. Our construction is divided into three stages as follows:

1) *LWE to Intermediate Ciphertexts:* The goal of this initial stage is to reinterpret each LWE ciphertext as a *larger* intermediate representation (reminiscent of a Module-LWE ciphertext [13], [49]). Crucially, this intermediate ciphertext encrypts the original LWE message as a *constant term polynomial* and retains necessary homomorphic properties for packing. This transformation is important because a) the constant term message enables straightforward aggregation of multiple such ciphertexts (Stage 2) and b) a carefully designed "correlated structure" in these intermediate ciphertexts enables our construction to ultimately rely on just two key-switching matrices (Stage 3).
2) *Aggregation of Intermediate Ciphertexts:* Stage 2 leverages homomorphic properties of the transformed ciphertexts to combine multiple such ciphertexts into one consolidated ciphertext. Since each transformed ciphertext embeds the original LWE message as a constant term polynomial, the aggregation becomes straightforward. The aggregated ciphertext's plaintext polynomial embeds the input LWE messages into distinct coefficients.
3) *Conversion to RLWE Ciphertext:* The final stage converts the intermediate ciphertext (which now holds all packed messages) into a standard, compact RLWE ciphertext. This is achieved via an iterative key-switching procedure that "collapses" the components, efficiently utilizing the correlated structure and Galois automorphic images of two elementary key-switching matrices.

At first, the transformation of LWE ciphertexts into significantly larger intermediate ciphertext in the initial stage might appear counterintuitive to achieving efficiency. However, the key to this approach lies in the nature of these expanded structures. The bulk of the data within these

intermediate forms is determined solely by the fixed random components of the original LWE ciphertexts and the key-switching matrices. In the CRS model where these random elements are fixed and known beforehand, the computationally intensive operations involved in constructing and processing these larger structures can be mainly moved to an offline preprocessing phase. This idea of investing more in offline computation is a recurring principle across all the three stages of our packing algorithm. It allows a substantial reduction in online computation time and minimizes overhead associated with cryptographic materials during the online packing process.

Stage 1: LWE to Intermediate Ciphertext. The first stage transforms each LWE input ciphertext into a specially structured ciphertext that resembles a Module-LWE (MLWE) ciphertext [13], [49]. Specifically, we will construct an intermediate ciphertext where the random component and the secret key are *modules* over the ring $R_q = \mathbb{Z}_q[X]/(X^d+1)$. The pseudorandom component is an element in R_q and will embed the original LWE message as a constant term polynomial, masked by an inner product between the random component and the secret key. However, the crucial difference from a typical MLWE ciphertext is the carefully designed correlations among the secret key components. Looking ahead, this correlated structure of the secret key components is essential for enabling the use of two key-switching matrices in Stage 3.

The core idea begins with embedding the input LWE ciphertext, $(\mathbf{a}, b = -\langle \mathbf{a}, \mathbf{s}\rangle + m)$, as an RLWE ciphertext $(\tilde{a}, \tilde{b} = -\tilde{a}\tilde{s} + \tilde{m})$ as in CDKS [18] (see Eq. 1). However, as discussed previously, the non-zero coefficients of the embedded plaintext message need to be zeroed out to enable packing. CDKS handles this by incrementally merging the ciphertexts in a recursive fashion, only freeing up necessary coefficient slots required for that stage of merging.

Instead, our insight is to transform the RLWE interpretation $(\tilde{a}, \tilde{b})$ into a carefully designed intermediate representation *upfront* that is more amenable to packing. Specifically, we aim to construct an intermediate ciphertext that embeds the original LWE message as a constant term polynomial, e.g. message of the form $\hat{m}(X) = m$. Looking ahead, this "sparsity" of the message allows multiple such ciphertexts to be aggregated without interference between messages.

To achieve this, we leverage the following formulation of the trace function (defined as the sum of Galois conjugates [33]) expressed in terms of two generators of the Galois group. We recall that for the cyclotomic ring R in this work, the Galois group is isomorphic to the additive group $\mathbb{Z}_{d/2} \times \mathbb{Z}_2$ [33]. τ_g and τ_h in Lemma 1 corresponds to the two generators $(1, 0)$ and $(0, 1)$ in $\mathbb{Z}_{d/2} \times \mathbb{Z}_2$. We leave the proof to Appendix B.4.

Lemma 1. *Let $p(X) \in \mathbb{Z}[X]/(X^d + 1)$ such that $p(X) = \sum_{i=0}^{d-1} c_i X^i$ where d is a power of two. Let $g = 5$ and $h = 2d - 1$, and define $Tr : R \to R$ as*

$$Tr(p) := \sum_{j=0}^{d/2-1} \tau_g^j(p) + \tau_h \circ \tau_g^j(p).$$

Then $Tr(p) = d \cdot c_0$.

The key insight to forming the intermediate ciphertext lies in how its components are constructed from the initial RLWE-like interpretation $(\tilde{a}, \tilde{b})$. The first (random) component of the new intermediate ciphertext is a vector of d polynomials $\hat{\mathbf{a}} \in R_q^d$. The first half of $\hat{\mathbf{a}}$ is defined as $\hat{\mathbf{a}}[j] = d^{-1} \cdot \tau_g^j(\tilde{a})$ for $j < d/2$. The second half is the automorphic image of the first half by τ_h, e.g. $\hat{\mathbf{a}}[d/2 : d] = \tau_h(\hat{\mathbf{a}}[0 : d/2])$.

The original $\tilde{b}$ is re-interpreted to serve as the pseudorandom component of this new intermediate ciphertext. Specifically, suppose we define the corresponding intermediate secret key vector (module) as $\hat{\mathbf{s}} \in R_q^d$ where the first half is $\hat{\mathbf{s}}[j] = \tau_g^j(\tilde{s})$ and the second half is the automorphic image of the first half by τ_h, e.g. $\hat{\mathbf{s}}[d/2 : d] = \tau_h(\hat{\mathbf{s}}[0 : d/2])$. Then, the component $\tilde{b}$ satisfies:

$$\tilde{b} = -\langle \hat{\mathbf{a}}, \hat{\mathbf{s}}\rangle + \hat{m} \pmod q \tag{2}$$

We note that this equation is derived from the application of the operator Tr from Lemma 1 appropriately on Eq. 1 (detailed in Appendix B.1). In particular, the pair $(\hat{\mathbf{a}}, \tilde{b})$ forms an MLWE-like ciphertext encrypting message $\hat{m}(X) = m$.

Importantly, the structure $(\hat{\mathbf{a}}, \tilde{b})$ maintains necessary homomorphic properties (akin to MLWE) for packing. It supports plaintext absorption (multiplication to each component) and homomorphic addition (component-wise addition), which is crucial for Stage 2.

In summary, Stage 1 converts an LWE ciphertext $(\mathbf{a}, b)$ (encrypting m) into an intermediate ciphertext encrypting $\hat{m}(X) = m$, which we denote as $\mathsf{IRCtx}(\hat{m})$ (stands for Intermediate Representation CipherTeXt).

We present the formal pseudocode of this transformation in Algorithm 1's subroutine TRANSFORM. See Appendix B.1 for further detailed mathematical derivation of $\mathsf{IRCtx}(\hat{m})$.

Stage 2: Aggregation of Intermediate Ciphertexts. Stage 2 combines d IRCtx ciphertexts (from Stage 1) into a single IRCtx, packing their messages as distinct coefficients.

Consider d input LWE ciphertexts: $(\mathbf{a}_0, b_0)$, $\ldots$, $(\mathbf{a}_{d-1}, b_{d-1})$, where the k-th ciphertext encrypts m_k. Applying Stage 1's TRANSFORM to each $(\mathbf{a}_k, b_k)$ yields: $\mathsf{IRCtx}(\hat{m}_k) \leftarrow \text{TRANSFORM}(\mathbf{a}_k, b_k)$, where $\hat{m}_k(X) = m_k$ is a constant term.

We then multiply each $\mathsf{IRCtx}(\hat{m}_k)$ by X^k and sum the results. Leveraging the homomorphic properties of IRCtx, we obtain the following:

$$\sum_{k=0}^{d-1} \mathsf{IRCtx}(\hat{m}_k) \cdot X^k = \mathsf{IRCtx}(\sum_{k=0}^{d-1} \hat{m}_k X^k).$$

We denote this aggregated plaintext message as $\hat{m}_{agg} := \sum_{k=0}^{d-1} \hat{m}_k X^k$ and the corresponding ciphertext encrypting it

$\mathsf{IRCtx}(\hat{m}_{agg}) = (\hat{\mathbf{a}}_{agg}, \tilde{b}_{agg})$. The above operation positions the original plaintext messages in the LWE ciphertexts, m_k, into the coefficients of $\hat{m}_{agg}$. However, the intermediate ciphertext produced at this stage, $\mathsf{IRCtx}(\hat{m}_{agg})$, is considerably large, comprising $d+1$ elements from the ring R_q. The formal pseudocode of this stage is provided in Algorithm 1's PACK procedure, Line 4.

Stage 3: Conversion to RLWE Ciphertext. The aggregated intermediate ciphertext $\mathsf{IRCtx}(\hat{m}_{agg}) = (\hat{\mathbf{a}}_{\mathbf{agg}}, \tilde{b}_{agg}) \in R_q^d \times R_q$ is currently represented by $d+1$ ring elements in R_q and is effectively encrypted under a vector of secret key shares $\hat{\mathbf{s}}$ with $\hat{\mathbf{s}}[j] = \tau_g^j(\tilde{s})$ and $\hat{\mathbf{s}}[j+d/2] = \tau_h \circ \tau_g^j(\tilde{s})$ for $j < d/2$.

As a reminder, $\tilde{b}_{agg}$ can be expressed as (from Eq. 2):

$$\begin{aligned}\tilde{b}_{agg} &= -\langle \hat{\mathbf{a}}_{agg}, \hat{\mathbf{s}} \rangle + \hat{m}_{agg} \pmod q \\ &= -\left(\sum_{j=0}^{d/2-1} \hat{\mathbf{a}}_{agg}[j] \cdot \tau_g^j(\tilde{s}) \right) \pmod q \\ &\quad -\left(\sum_{j=0}^{d/2-1} \hat{\mathbf{a}}_{agg}[j+d/2] \cdot \left(\tau_h \circ \tau_g^j(\tilde{s})\right) \right) + \hat{m}_{agg} \pmod q.\end{aligned}$$

To obtain a standard two-component RLWE ciphertext encrypted under a single base secret key $\tilde{s}$, we must reduce the number of these components by re-expressing the masking part $\langle \hat{\mathbf{a}}_{\mathbf{agg}}, \hat{\mathbf{s}} \rangle$ with respect to a single base secret key $\tilde{s}$.

At a high level, this reduction is achieved through an iterative process that systematically "collapses" the components. In each step, a RLWE key-switching procedure (see Section 2 for full description) is applied, which re-expresses the pseudorandom component originally masked by one secret key share $\tau_g^k(\tilde{s})$ (resp. $\tau_h \circ \tau_g^k(\tilde{s})$) so that its masking now relies on a different share $\tau_g^{k-1}(\tilde{s})$ (resp. $\tau_h \circ \tau_g^{k-1}(\tilde{s})$), effectively reducing the number of distinct key shares in the overall ciphertext masking. This iterative process eventually consolidates all parts under two key shares $\tilde{s}$ and $\tau_h(\tilde{s})$.

This process relies on a set of key-switching matrices, but all necessary key-switching matrices are automorphic images of a single elementary key-switching matrix $\mathbf{K}_g = \mathsf{KS.Setup}(\tau_g(\tilde{s}), \tilde{s})$. This is because if $\mathbf{K}_g$ transforms a ciphertext component from being encrypted under $\tau_g(\tilde{s})$ to $\tilde{s}$, then its automorphic image $\tau_g^k(\mathbf{K}_g)$ (resp. $\tau_h \circ \tau_g^k(\mathbf{K}_g)$) will transform a component from $\tau_g^{k+1}(\tilde{s})$ to $\tau_g^k(\tilde{s})$ (resp. $\tau_h \circ \tau_g^{k+1}(\tilde{s})$ to $\tau_h \circ \tau_g^k(\tilde{s})$).

This creates a "telescoping" effect where each step in the reduction uses a key related to $\mathbf{K}_g$ by an appropriate Galois automorphism, allowing the component reduction to be performed efficiently using variants of just one base key-switching matrix. After $d-2$ such iterations, the number of random components of $\mathsf{IRCtx}(\hat{m}_{agg})$ is reduced to two, with the resultant ciphertext encrypted under two secret key shares $\tilde{s}$ and $\tau_h(\tilde{s})$. To eliminate the key share $\tau_h(\tilde{s})$, we perform the final key-switching using $\mathbf{K}_h = \mathsf{KS.Setup}(\tau_h(\tilde{s}), \tilde{s})$. The final result is a standard two-component RLWE ciphertext (a_{fin}, b_{fin}), where $b_{fin} = -a_{fin} \cdot \tilde{s} + \hat{m}_{agg} + e_{tot}$ and e_{tot} is the total accumulated noise from the key-switchings.

The formal pseudocode of this stage is presented in Algorithm 1's subroutine COLLAPSE. The full details of this iterative component reduction algorithm, including the specific key-switching steps, are provided in Appendix B.2.

Putting It Together. The discussions in the preceding sections can be summarized as the unified algorithm InspiRING.Pack. The pseudocode is in Algorithm 1. We remind the readers that the highlighted parts of the algorithm correspond to the preprocessable components in the CRS model (Section 2.2. We denote $\tilde{s}$ is the natural polynomial interpretation of the LWE secret $\mathbf{s}$.

Algorithm 1 InspiRING algorithm

1: **procedure** PACK($[\mathbf{A}, \mathbf{b}] \in \mathbb{Z}_q^{d \times (d+1)}, \mathbf{K}_g = [\mathbf{w}_g, \mathbf{y}_g], \mathbf{K}_h = [\mathbf{w}_h, \mathbf{y}_h] \in R_q^{\ell \times 2}$)
2: **for** $k = 0 \dots d-1$ **do**
3: $(\hat{\mathbf{a}}_k, \tilde{b}_k) \leftarrow$ TRANSFORM($\mathbf{A}[k], \mathbf{b}[k]$)
4: $(\hat{\mathbf{a}}_{\mathbf{agg}}, \tilde{b}_{agg}) \leftarrow \sum_{k=0}^{d-1} (\hat{\mathbf{a}}_k, \tilde{b}_k) \cdot X^k$
5: $(a_{fin}, b_{fin}) \leftarrow$ COLLAPSE($(\hat{\mathbf{a}}_{agg}, \tilde{b}_{agg}), [\mathbf{w}_g, \mathbf{y}_g], [\mathbf{w}_h, \mathbf{y}_h]$)
6: **return** $(a_{fin}, b_{fin}) \in R_q \times R_q$

— Subroutines —

1: **procedure** TRANSFORM($(\mathbf{a}, b) \in \mathbb{Z}_q^d \times \mathbb{Z}_q$)
2: $\tilde{a} \leftarrow d^{-1} \sum_{i=0}^{d-1} \mathbf{a}[i] X^{-i}$ ▷ $d \cdot d^{-1} = 1 \mod q$
3: $\tilde{b} \leftarrow b$
4: Initialize $\hat{\mathbf{a}} \in R_q^d$
5: **for** $j = 0 \dots d/2 - 1$ **do**
6: $\hat{\mathbf{a}}[j] \leftarrow \tau_g^j(\tilde{a})$
7: $\hat{\mathbf{a}}[j + d/2] \leftarrow \tau_h \circ \tau_g^j(\tilde{a})$
8: **return** $(\hat{\mathbf{a}}, \tilde{b}) \in R_q^d \times R_q$

1: **procedure** COLLAPSE($(\hat{\mathbf{a}}_{agg}, \tilde{b}_{agg}) \in R_q^d \times R_q, \mathbf{K}_g = [\mathbf{w}_g, \mathbf{y}_g] \in R_q^{\ell \times 2}, \mathbf{K}_h = [\mathbf{w}_h, \mathbf{y}_h] \in R_q^{\ell \times 2}$)
2: $\hat{\mathbf{a}}_1 \leftarrow \hat{\mathbf{a}}_{agg}[0 : d/2]$
3: $\hat{\mathbf{a}}_2 \leftarrow \hat{\mathbf{a}}_{agg}[d/2 : d]$
4: $(a_1, b_1) \leftarrow$ COLLAPSEHALF($(\hat{\mathbf{a}}_1, \tilde{b}_{agg}, [\mathbf{w}_g, \mathbf{y}_g], I)$)
5: $(a_2, b_2) \leftarrow$ COLLAPSEHALF($(\hat{\mathbf{a}}_2, b_1, [\mathbf{w}_g, \mathbf{y}_g], \tau_h)$)
6: $(a, b) \leftarrow$ COLLAPSEONE($([a_1, a_2], b_2), [\mathbf{w}_h, \mathbf{y}_h]$)
7: **return** $(a, b) \in R_q \times R_q$

1: **procedure** COLLAPSEHALF($(\hat{\mathbf{a}}_{half}, \tilde{b}_{half}) \in R_q^{d/2} \times R_q, \mathbf{K}_g = [\mathbf{w}_g, \mathbf{y}_g] \in R_q^{\ell \times 2}, \rho \in \{I, \tau_h\}$)
2: Rename $(\hat{\mathbf{a}}_{half}, \tilde{b}_{half})$ as $(\mathbf{a}^{(d/2-1)}, b^{(d/2-1)})$
3: **for** $k = d/2 - 1 \dots 0$ **do**
4: $\mathbf{K}_g^{k-1} \leftarrow \rho \circ \tau_g^{k-1}(\mathbf{K}_g)$
5: $(\mathbf{a}^{(k-1)}, b^{(k-1)}) \leftarrow$ COLLAPSEONE($(\mathbf{a}^{(k)}, b^{(k)}), \mathbf{K}_g^{k-1}$)
6: **return** $(\mathbf{a}^{(0)}, b^{(0)}) \in R_q \times R_q$

1: **procedure** COLLAPSEONE($(\mathbf{a}, b) \in R_q^k \times R_q, \mathbf{K} = [\mathbf{w}, \mathbf{y}] \in R_q^{\ell \times 2}$)
2: $(a', b') \leftarrow \mathsf{KS.Switch}((\mathbf{a}[k-1], b), [\mathbf{w}, \mathbf{y}])$
3: $\mathbf{a}' \leftarrow [\mathbf{a}[0], \dots, \mathbf{a}[k-3], \mathbf{a}[k-2] + a']^\top \in R_q^{k-1}$
4: **return** $(\mathbf{a}', b') \in R_q^{k-1} \times R_q$

Preprocessing in the CRS Model. A key strength of InspiRING algorithm is its inherent suitability for preprocessing in the CRS model where the random components of the ciphertexts are fixed. As detailed in the preceding stages, a substantial portion of the data manipulated during the

packing process depends solely on the random components of the initial LWE ciphertexts and the key-switching matrices $\mathbf{K}_g$ and $\mathbf{K}_h$. This characteristic allows for significant portions of the computation to be performed offline, making the online packing phase very efficient.

The benefits of this approach are evident in the first two stages. In Stage 1, the random components of the resultant $\mathsf{IRCtx}(\hat{m}_k)$ ciphertexts are entirely determined by the random vectors of the input LWE ciphertexts. Similarly, in Stage 2, the bulk of operations involves combining these preprocessable random components. The online aspects of these stages are minimal: the re-interpretation of the LWE pseudorandom components b_k into their ring element counterparts $\tilde{b}_k$ requires no actual computation, and their aggregation into $\mathsf{IRCtx}(\hat{m}_{agg})$ is a straightforward process of assigning the constant $\tilde{b}_k$ values as polynomial coefficients.

The advantages extend to Stage 3, though perhaps less immediately obvious. This stage involves an iterative reduction of the intermediate ciphertext's components using key-switchings. After Stage 2, the $\mathsf{IRCtx}(\hat{m}_{agg})$ consists of d random components (which, as established, can be preprocessed offline) and a single pseudorandom polynomial component $\tilde{b}_{agg}$ (which must be processed online). The iterative key-switching procedure is designed to maintain an important invariant: throughout the reduction, the evolving random components remain dependent only on the initial, preprocessable random components of the input LWE ciphertexts and the key-switching matrices $\mathbf{K}_g$ and $\mathbf{K}_h$. This crucial property ensures that most data necessary to update the pseudorandom component during each step of the reduction can also be precomputed. Consequently, the online operations in Stage 3 are significantly efficient, contributing to the overall efficiency of the online packing algorithm.

Theorem 1. InspiRING *in the CRS model can pack d LWE ciphertexts in $O(d^3 + \ell d^2 \lg d)$ offline time and $O(\ell \cdot d^2)$ online time where ℓ is the dimension of the key-switching matrix.*

Furthermore, our algorithm also results in smaller noise growth (both asymptotically and practically).

Theorem 2. *Let the error distribution χ be subgaussian with parameter σ_χ. Let ℓ be the dimension of the key-switching matrix and z be the decomposition base. Under the independence heuristic,* InspiRING *incurs an additive noise $e_{pack} \in R_q$, which has subgaussian coefficients with parameter σ_{pack} and $\sigma_{pack}^2 \leq \ell d^2 z^2 \sigma_\chi^2/4$.*

Our packing scheme has the same asymptotic running time, but much faster concrete running time as well as smaller noise growth compared to CDKS [18]. We corroborate our analysis with experimental evaluation in Appendix B.3.

Security. Beyond RLWE hardness, our packing scheme relies on the standard *circular security* assumption, as key-switching matrices encrypt (scaled) automorphic images of the secret key. This assumption is common in lattice based FHE literature [37], [14], [13], [12], [18], [44], [7] and PIR [66], [61], [16], [62], [52], [4].

3.3. InsPIRe$_0$: PIR from Ring Packing

A direct application of InspiRING to PIR results in our first PIR construction InsPIRe$_0$. InsPIRe$_0$ is instantiated on top of DoublePIR by using InspiRING to pack the result of the DoublePIR responses. This protocol is useful when the entry size is small (e.g. 1 bit) and when optimizing for specific metrics such as runtime, as we show in the evaluation. We defer further description to the full paper.

4. InsPIRe: PIR from Ring Packing and Homomorphic Polynomial Evaluation

In this section, we present InsPIRe, our low query communication and high-throughput PIR protocol that uses our InspiRING ring packing with preprocessing algorithm as a building block. One can follow the prior PIR frameworks such as YPIR [62] and replace their ring packing algorithms with InspiRING. This immediately results in an improved PIR scheme. However, we will present a new PIR protocol that both leverages InspiRING as well as polynomial evaluation to obtain further communication improvements.

We start by presenting an overview of the prior framework using ring packing and highlight some limitations. Initially, suppose the database consists of N entries, each as an element in $\mathbb{Z}_p^d$. Note, the choice of using d here is for simplicity and we will generalize later. We introduce an additional configurable parameter t and model the plaintext database as a matrix $\mathbf{D} \in \mathbb{Z}_p^{td \times N/t}$. Each column of the matrix corresponds to t database entries of $\mathbb{Z}_p^d$.

During the query phase, the client queries for the column i that contains the desired entry. To achieve this, the client generates LWE encryptions of a one-hot selection vector of length N/t. All ciphertexts encrypt zero except the i-th ciphertext which encrypts one. The N/t LWE ciphertexts can be represented as the rows of the matrix $[\mathbf{A}, \mathbf{b}] \in \mathbb{Z}_q^{N/t \times (d+1)}$ which are sent to the server. Next, the server performs matrix-matrix multiplication to obtain $\mathbf{D} \cdot [\mathbf{A}, \mathbf{b}]$, which yields td LWE encryptions of the i-th column of $\mathbf{D}$. At this point, each consecutive chunk of d LWE ciphertexts corresponds to a database entry over $\mathbb{Z}_p^d$. Finally, the server uses ring packing (such as InspiRING) to translate each chunk of d LWE ciphertexts into a single RLWE ciphertext before returning the resulting t RLWE ciphertexts back to the client.

In this scheme, the server returns encryptions corresponding to the entire column of t entries. To reduce the response size, we ideally want the server to only return the desired entry instead. Previous approaches, such as OnionPIR [66] and Spiral [61], employed encryptions of one-hot selection vectors for this step. However, these methods typically require $\Theta(\lg(t))$ ciphertexts or large evaluation keys, adding significant communication.

4.1. Homomorphic Polynomial Evaluation

We present a new PIR protocol that introduces a new way of encoding the plaintext database. Instead of explicitly representing each column as a concatenation of t database entries, our key idea is to implicitly represent it as coefficients of a polynomial that *evaluates* to the entries in that column for some publicly fixed evaluation points.

With this new database encoding, the initial homomorphic column selection will obtain encrypted polynomial coefficients. Afterwards, we use homomorphic polynomial evaluation to extract the desired entry. By carefully constructing the polynomial, we can perform the evaluation using a *single* RGSW ciphertext. Furthermore, our design utilizes RLWE-RGSW external products (see Section 2) to minimize noise growth during homomorphic evaluation. By choosing the evaluation points appropriately, our homomorphic evaluation will only incur *additive* noise growth for each multiplication.

Polynomials for Database Encoding. We proceed more formally to define the polynomial representations for each database column. For convenience, we will assume that N, and t are powers of two, but we will generalize later. Recall that our plaintext database is modeled as $\mathbf{D} \in \mathbb{Z}_p^{td \times N/t}$ where each column contains t entries from $\mathbb{Z}_p^d$. We denote $\mathbf{y}_j^{(i)} \in \mathbb{Z}_p^d$ as the j-th entry in the i-th column.

We will use polynomials to represent each column of the matrix $\mathbf{D}$. We first begin by re-interpreting each $\mathbf{y}_j^{(i)} \in \mathbb{Z}_p^d$ as an element in $R_p = \mathbb{Z}_p[X]/(X^d + 1)$ by mapping the d elements in $\mathbb{Z}_p$ as the polynomial coefficients. We will denote these re-interpretations as $y_j^{(i)} \in R_p$.

Let $R_p[Z]$ be the polynomial ring over R_p. Imagine that we have t distinct points $z_0, \ldots, z_{t-1} \in R_p$ that are fixed publicly beforehand. We will pick these t points later. For each column i, suppose there exists a polynomial $h^{(i)}(Z) = \sum_{j=0}^{t-1} c_j^{(i)} Z^j \in R_p[Z]$ such that the following holds:

$$h^{(i)}(z_j) = y_j^{(i)}, \forall j \in \{0, 1, \ldots, t-1\}.$$

Suppose the server can compute the coefficients $c_0^{(i)}, \ldots, c_{t-1}^{(i)} \in R_p$ of the polynomial $h^{(i)}(Z)$ for each column i. We construct encoded database $\mathbf{D}' \in \mathbb{Z}_p^{td \times N/t}$, where each column i consists of the corresponding polynomial coefficients $c_j^{(i)}$ interpreted as elements in $\mathbb{Z}_p^d$.

Homomorphic Polynomial Evaluation. Suppose the client wants to retrieve the k-th entry from the i-th column, $y_k^{(i)}$. Consider the point after the server has performed initial homomorphic selection of column i *on the encoded database* $\mathbf{D}'$. At this point, the server holds t RLWE encryptions of the coefficients $c_0^{(i)}, \ldots, c_{t-1}^{(i)}$ of the polynomial $h^{(i)}(Z)$. Since the evaluation points z_j are publicly fixed and the polynomials $h^{(i)}(Z)$ are constructed such that $h^{(i)}(z_j) = y_j^{(i)}$ for each $0 \leq j < t$, the client can send a *single* RGSW ciphertext encrypting the evaluation point z_k to the server. The server can then homomorphically evaluate the polynomial $h^{(i)}(Z)$ using the RLWE encrypted coefficients $c_j^{(i)}$ and a single RGSW encrypted evaluation point z_k. This would obtain the desired entry $h^{(i)}(z_k) = y_k^{(i)}$. To perform evaluation, we will use the Horner-style method (see EVALPOLY in Algorithm 4), that only involves RLWE-RGSW external products and RLWE additions.

So far, we have not discussed how the evaluation points z_j are chosen. For our PIR protocol to be efficient, we want the evaluation points z_j to satisfy the following properties:

1) *Noise Management during Evaluation:* Homomorphic polynomial evaluation involves large-depth multiplications (e.g. in Horner's method), each increasing the noise in the resulting ciphertext. We want to strategically choose the evaluation points so that the noise growth from multiplications is minimal (ideally, additive).
2) *Interpolation in Rings:* The evaluation points must guarantee a polynomial interpolation within the ring R_p. Since R_p is not necessarily a field, the standard conditions for interpolation (requiring distinct points) are not sufficient. We need to ensure the existence of the interpolating polynomial for any set of database entries.

Unit Monomials as Evaluation Points. To address the challenges of noise management and interpolation, we propose a specific choice for the evaluation points. Our key insight is to select evaluation points $z_0, \ldots, z_{t-1}$ that are both *unit monomials* and *roots of unity*. Here, we will define unit monomials as elements in R_p of the form $\pm X^k$ where $0 \leq k < d$. If the RGSW ciphertext encrypting the evaluation point encrypts a unit monomial, the external product operation incurs only *additive noise* growth, rather than multiplicative.

Lemma 2. *[Theorem 2.19 [61], adapted] Given* $\mathsf{RLWE}(m_0)$*, let* $\mathsf{RGSW}(m_1)$ *be a fresh ciphertext encrypting a unit monomial, e.g.* $m_1 = \pm X^k$ *for some* $0 \leq k < d$*. Let the error distribution* χ *be subgaussian with parameter* σ_χ*, and let* ℓ *and* z *be the gadget parameters of the RGSW ciphertext. Under the independence heuristic, the external product* $\mathsf{RLWE}(m_0) \boxdot \mathsf{RGSW}(m_1)$ *incurs additive noise* $e_{ep} \in R_q$ *whose coefficients are subgaussian with parameter* σ_{ep} *and* $\sigma_{ep}^2 \leq \ell d z^2 \sigma_\chi^2 / 2$*.*

By ensuring that all $z_0, \ldots, z_{t-1}$ are unit monomials, the entire homomorphic evaluation process (involving $t-1$ multiplications and additions) incurs only additive noise.

Next, we discuss how to guarantee polynomial interpolation within R_p using these unit monomial evaluation points. To guarantee interpolation, our key insight is to use powers of a *primitive t-th root of unity* ω as the evaluation points. Specifically, for $t \leq 2d$ a power of two, we will choose a primitive t-th root $\omega = X^{2d/t}$. With this choice, we define the evaluation points as powers of ω: $z_k := \omega^k$ for $k = 0 \ldots t-1$. Crucially, we see that all the evaluation points z_k are unit monomials. These points are distinct and the Vandermonde matrix (e.g. DFT matrix) formed by them is invertible over R_p, assuming p is odd [15]. Essentially, the polynomial interpolation then amounts to a matrix-vector product between the corresponding IDFT matrix and the

vector of $y_j^{(i)}$s, which can be done efficiently using Cooley-Tukey style FFT algorithm.

Constraints and Generalizations. This specific construction imposes two main constraints:

- $t \leq 2d$: This is required for the evaluation points to be unit monomials. While our experimental evaluations indicate that $t \ll 2d$ often suffices for effective PIR, scenarios requiring larger folding factor ($t > 2d$) are possible. We discuss extensions using multivariate polynomial interpolation in the full paper.
- t **is a power of two:** This is required for the Cooley-Tukey style FFT algorithm used for the interpolation during database encoding. If t is not a power of two, a computationally less efficient Lagrange interpolation can be used instead. We present this Lagrange interpolation based method in the full paper.

4.2. Putting It Together

We present our PIR protocol InsPIRe = (Setup, Query, Respond, Extract) using InspiRING and homomorphic polynomial evaluation in Algorithms 2, 3, 4, 5. For convenience, we assume that the public parameters pp are globally available to the subroutines of the PIR algorithms and do not explicitly specify them as inputs. For instance, we assume InspiRING uses the RLWE parameters rp and the gadget parameters gp_{ks}. Note that GenKsMatY() in Algorithm 3 is essentially the same as KS.Setup except it only generates the pseudorandom component of the key-switching matrix.

Preprocessing in the CRS Model. We remind the readers that the highlighted parts of InsPIRe.Respond correspond to the preprocessable parts of the algorithm prior to serving queries (see Section 2.2). These correspond to parts that only depend on the outputs from InsPIRe.Setup. We note that this extends to InspiRING.Pack as well (Algorithm 1).

Generalizing to Arbitrary Entry Size. The pseudocode assumes that each entry is encoded as $\mathbb{Z}_p^d$ of length d. To support arbitrary length m, we use standard reductions:

- $m > d$**:** Divide each entry into m/d chunks. Construct m/d databases $\in \mathbb{Z}_q^{td \times N/t}$ where the i-th database contains the i-th chunk. In the Respond algorithm, the server processes the client's query on all databases.
- $m < d$**:** Bundle d/m entries as a single entry over $\mathbb{Z}_p^d$. The client queries for the bundle with the desired entry.

Theorem 3. *Let the error distribution χ be subgaussian with parameter σ_χ. Let ℓ_{ks} and z_{ks} be the gadget parameters of the key-switching matrices. Let ℓ_{gsw} and z_{gsw} be the gadget parameters of the RGSW ciphertext. The error polynomial of the RLWE ciphertext* ct *has the form $e = e_{main} + e_{overflow} \in R_q$. Under the independence heuristics, e_{main} has subgaussian coefficients with parameter $\sigma_{main}^2 \leq Np^2\sigma_\chi^2 + t\ell_{ks}d^2z_{ks}^2\sigma_\chi^2/4 + t\ell_{gsw} \cdot dz_{gsw}^2 \cdot \sigma_\chi^2/2$ and $||e_{overflow}||_\infty \leq tp/2$.*

The $e_{overflow}$ term is introduced from the homomorphic polynomial evaluation because the sum of the embedded plaintexts typically overflows the R_p plaintext space. However, Theorem 3 shows that this additional $e_{overflow}$ is negligible in practice. We leave the proof to the full paper.

Theorem 4. *For subgaussian χ with parameter σ_χ, define $\bar{\sigma}_{main}^2 = Np^2\sigma_\chi^2 + t\ell_{ks}d^2z_{ks}^2\sigma_\chi^2/4 + t\ell_{gsw} \cdot dz_{gsw}^2 \cdot \sigma_\chi^2/2$. Then, for $\delta > 2d\exp\left(-\pi(\Delta/2 - tp/2)^2/\bar{\sigma}_{main}^2\right)$,* InsPIRe *is $(1-\delta)$-correct.*

Algorithm 2 InsPIRe.Setup

procedure Setup($1^\lambda, \mathbf{D} \in \mathbb{Z}_p^{td \times N/t}$)
  $\mathsf{rp} \leftarrow (d(\lambda), q(\lambda), \chi(\lambda), p)$ as the RLWE params
  $\mathsf{gp}_{ks} \leftarrow (z_{ks}, \ell_{ks})$ as the gadget params for the KS matrices
  $\mathsf{gp}_{gsw} \leftarrow (z_{gsw}, \ell_{gsw})$ as the gadget params for the RGSW ciphertext
  $\mathsf{qry}_{\mathsf{off}} \leftarrow$ GenFixedQueryParts()
  $\mathsf{pp} \leftarrow (N, t, \mathsf{rp}, \mathsf{gp}_{ks}, \mathsf{gp}_{gsw}, \mathsf{qry}_{\mathsf{off}})$
  $\mathbf{D}' \leftarrow$ EncodeDB($\mathbf{D}$)
  return $(\mathsf{pp}, \mathbf{D}')$

procedure EncodeDB($\mathbf{D} \in \mathbb{Z}_p^{td \times N/t}$)
  Initialize $\mathbf{D}' \in \mathbb{Z}_p^{td \times N/t}$
  for $i = 0 \ldots N/t - 1$ **do**
    $y_j^{(i)} \leftarrow j$-th element in column i interpreted as R_p for $0 \leq j < t$
    $[c_0^{(i)}, \ldots, c_{t-1}^{(i)}]^\top \leftarrow$ Interpolate($[y_0^{(i)}, \ldots, y_{t-1}^{(i)}]^\top$)
    Cast $[c_0^{(i)}, \ldots, c_{t-1}^{(i)}]^\top$ as a vector over $\mathbb{Z}_p^{td}$, and embed into $\mathbf{D}'[:][i]$
  return $\mathbf{D}'$

procedure Interpolate($[y_0, \ldots, y_{t-1}]^\top \in R_p^t$)
  $\mathbf{c} \leftarrow$ CooleyTukey($[y_0, \ldots, y_{t-1}]^\top$)
  return $t^{-1} \cdot \mathbf{c}$

procedure CooleyTukey($[y_0, \ldots, y_{m-1}]^\top \in R_p^m$)
  if $m = 1$ **then**
    return $[y_0]^\top$
  $\mathbf{c}_{even} \leftarrow$ CooleyTukey($[y_0, y_2, \ldots, y_{m-2}]^\top$)
  $\mathbf{c}_{odd} \leftarrow$ CooleyTukey($[y_1, y_3, \ldots, y_{m-1}]^\top$)
  Initialize $\mathbf{c} \in R_p^m$
  for $i = 0 \ldots m/2 - 1$ **do**
    $\omega \leftarrow X^{-2d/m \cdot i}$
    $\mathbf{c}[i] \leftarrow \mathbf{c}_{even}[i] + \omega \cdot \mathbf{c}_{odd}[i]$
    $\mathbf{c}[i + m/2] \leftarrow \mathbf{c}_{even}[i] - \omega \cdot \mathbf{c}_{odd}[i]$
  return $\mathbf{c}$

procedure GenFixedQueryParts()
  $\mathbf{A} \leftarrow U(\mathbb{Z}_q^{N/t \times d})$
  $\mathbf{w}_g \leftarrow U(R_q^{\ell_{ks}})$
  $\mathbf{w}_h \leftarrow U(R_q^{\ell_{ks}})$
  return $\mathsf{qry}_{\mathsf{off}} = (\mathbf{A}, \mathbf{w}_g, \mathbf{w}_h)$

Algorithm 3 InsPIRe.Query

procedure Query(pp, idx)
  Parse pp as $(N, t, \mathsf{rp}, \mathsf{gp}_{ks}, \mathsf{gp}_{gsw}, \mathsf{qry}_{\mathsf{off}})$
  Parse $\mathsf{qry}_{\mathsf{off}}$ as $(\mathbf{A}, \mathbf{w}_g, \mathbf{w}_h)$
  Parse idx as (i, j)
  $\mathbf{s} \leftarrow \chi(\mathbb{Z}^d)$
  $\tilde{s} \leftarrow \sum_{k=0}^{d-1} \mathbf{s}[k] \cdot X^k$
  Initialize $\mathbf{b} \in \mathbb{Z}_q^{N/t}$
  for $k = 0 \ldots N/t - 1$ **do**
    $m_k \leftarrow k = i$
    $e_k \leftarrow \chi(\mathbb{Z})$
    $\mathbf{b}[k] \leftarrow -\langle \mathbf{A}[k], \mathbf{s} \rangle + e_k + \Delta m_k$ ▷ $\Delta = \lfloor q/p \rceil$
  $\omega \leftarrow X^{2d/t}$
  $\mathsf{RGSW}(\omega^j) \leftarrow \mathsf{RGSW.Enc}(\tilde{s}, \omega^j)$
  $\mathbf{y}_g \leftarrow$ GenKsMatY($\tilde{s}, \mathbf{w}_g, \tau_g$)
  $\mathbf{y}_h \leftarrow$ GenKsMatY($\tilde{s}, \mathbf{w}_h, \tau_h$)
  $\mathsf{qry} \leftarrow (\mathbf{b}, \mathsf{RGSW}(\omega^j), \mathbf{y}_g, \mathbf{y}_h)$
  return $(\mathsf{st} = \tilde{s}, \mathsf{qry})$

procedure GenKsMatY($\tilde{s}, \mathbf{w}, \rho \in \{\tau_g, \tau_h\}$)
  $\mathbf{e} \leftarrow \chi(R_q^{\ell_{ks}})$
  $\mathbf{y} \leftarrow \tilde{s}\mathbf{w} + \mathbf{e} + \rho(\tilde{s}) \cdot \mathbf{g}_{z_{ks}}$
  return $\mathbf{y}$

Algorithm 4 InsPIRe.Respond

procedure RESPOND(pp, $\mathbf{D}' \in \mathbb{Z}_q^{td \times N/t}$, qry)
  Parse pp as $(N, t, \mathsf{rp}, \mathsf{gp}_{ks}, \mathsf{gp}_{gsw}, \mathsf{qry}_{\mathsf{off}})$
  Parse $\mathsf{qry}_{\mathsf{off}}$ as $(\mathbf{A}, \mathbf{w}_g, \mathbf{w}_h)$
  Parse qry as $(\mathbf{b}, \mathsf{RGSW}(\omega^j), \mathbf{y}_g, \mathbf{y}_h)$
  $\mathbf{K}_g \leftarrow [\mathbf{w}_g, \mathbf{y}_g]$
  $\mathbf{K}_h \leftarrow [\mathbf{w}_h, \mathbf{y}_h]$
  $[\mathbf{H}, \mathbf{b}'] \leftarrow \mathbf{D}' \cdot [\mathbf{A}, \mathbf{b}]$
  for $k = 0 \ldots t-1$ **do**
    $\mathbf{H}_k \leftarrow \mathbf{H}[kd : kd+d)][:]$
    $\mathbf{b}'_k \leftarrow \mathbf{b}'[kd : kd+d]$
    $(a_k, b_k) \leftarrow \mathsf{InspiRING.PACK}([\mathbf{H}_k, \mathbf{b}'_k], \mathbf{K}_g, \mathbf{K}_h)$
    Rename (a_k, b_k) to $\mathsf{RLWE}(c_k)$
  $\mathsf{resp} \leftarrow \mathrm{EVALPOLY}(\mathsf{RLWE}(c_0), \ldots, \mathsf{RLWE}(c_{t-1}), \mathsf{RGSW}(\omega^j))$
  return resp

procedure EVALPOLY($\mathsf{RLWE}(c_0), \ldots, \mathsf{RLWE}(c_{t-1}), \mathsf{RGSW}(\omega^j)$)
  $\mathsf{ct} \leftarrow \mathsf{RLWE}(c_{t-1})$
  for $k = t-2 \ldots 0$ **do**
    $\mathsf{ct} \leftarrow \mathsf{ct} \boxdot \mathsf{RGSW}(\omega^j) + \mathsf{RLWE}(c_k)$
  return ct

Algorithm 5 InsPIRe.Extract

procedure EXTRACT(pp, st, resp)
  Extract $\tilde{s}$ from st
  $y_j^{(i)} \leftarrow \mathsf{RLWE.Dec}(\tilde{s}, \mathsf{resp})$
  return $y_j^{(i)}$

Theorem 5. *Let ℓ_{ks} and ℓ_{gsw} be the gadget parameters of the key-switching matrices and the query RGSW ciphertext respectively. In the CRS model,* InsPIRe *runs in offline time $O(Nd^2 + t(d^3 + \ell_{ks}d^2 \lg d))$ and online time $O(Nd + t\ell_{ks}d^2 + t\ell_{gsw}d \lg(d))$.*

Theorem 6. *The total communication cost of* InsPIRe *is*

$$d\ell_{ks} \log_2 q + (N/t) \log_2 q + 4\ell_{gsw} d \log_2 q + 2d \log_2 q$$

Security. The security of our protocol relies on the RLWE hardness assumption and the circular security assumption for the key-switching matrices used in packing (see 3). We provide a proof sketch in Appendix A.2 and leave the formal proof to the full paper.

Comparison with $\mathsf{InsPIRe}_0$. Because $\mathsf{InsPIRe}_0$ is built on top of DoublePIR, it can only support small entry size such as 1 bit. On the other hand, InsPIRe is much more flexible and can support arbitrary entry size with very low communication overhead. As we show in Section 5, InsPIRe can even obtain lower communication than $\mathsf{InsPIRe}_0$, but the downside is the higher server runtime from packing during the initial homomorphic column selection. In contrast, $\mathsf{InsPIRe}_0$ packs the DoublePIR response consisting of a smaller number of LWE ciphertexts. In general, InsPIRe is the recommended choice unless the entry size is very small and server runtime is the primary metric to optimize.

5. Experimental Evaluation

In this section, we compare InsPIRe and $\mathsf{InsPIRe}_0$ with existing efficient PIR protocols in the CRS model. In particular, we do not compare with protocols that rely on client-specific cryptographic keys [4], [61], database-dependent hints [42] that need to be sent to the client beforehand, or streaming the database in an offline phase [80]. In our experimental evaluation, we are particularly interested in two metrics: the server's online runtime and the total communication required for a single PIR query.

InsPIRe introduces the interpolation degree (t) as a new parameter of the PIR protocol which can not be chosen in a simple optimization. To guide in choosing this parameters, we first show the effect of the interpolation degree on the performance of InsPIRe. In summary, we show that the choice of the interpolation degree results in a tradeoff between communication and computation. We compare InsPIRe with state-of-the-art PIR protocols, for small (1 bit), medium (64 B), and large (32 KB) entries.

Implementation & Evaluation Details. We implemented InspiRING with about 3,000 lines of Rust using building blocks for RLWE operations from spiral-rs [2]. Using InspiRING and building blocks from the YPIR implementation [3], we implemented InsPIRe in an additional 2,000 lines of Rust. Our code is publicly available on Github [4]. We perform all experimental evaluations on an Intel Xeon CPU @ 2.6 GHz, where we run all experiments in single-threaded mode for fair comparison with related work. We are primarily interested in two metrics, the total communication and the online server runtime. We provide breakdowns of the communication and runtime as well. In all experiments, offline runtimes are measured once and online runtimes are averaged over 5 runs with a standard deviation of less than 5%. For InsPIRe, we use lattice parameters in Table 1, which provide 128-bit security based on the lattice-estimator [2] and correctness parameter $\delta = 2^{-40}$ (see Appendix A.1).

TABLE 1: InsPIRe lattice parameters

d	$\log_2 q$	σ_χ	p	ℓ_{KS}	ℓ_{GSW}	z_{KS}	z_{GSW}
2048	56	6.4	65535	3	3	2^{19}	2^{19}

5.1. PIR Evaluation

Effect of Interpolation Degree. Figure 2 shows the performance metrics of InsPIRe for varying interpolation degrees. We use a 1 GB database to demonstrate the tradeoff, but the results apply to all other database sizes as well. For communication costs, we measure the upload and download costs, and break down the upload costs into the key material (the packing keys) and the query (the encrypted LWE indicator vector and RGSW ciphertext). We also break down the server's online time into three parts: the first matrix multiplication, the packing, and the polynomial evaluation.

For the server online runtime, Figure 2 shows that, for a fixed database size, a larger interpolation degree results

2. https://github.com/menonsamir/spiral-rs/
3. https://github.com/menonsamir/ypir/
4. https://github.com/google/private-membership/tree/main/research/InsPIRe

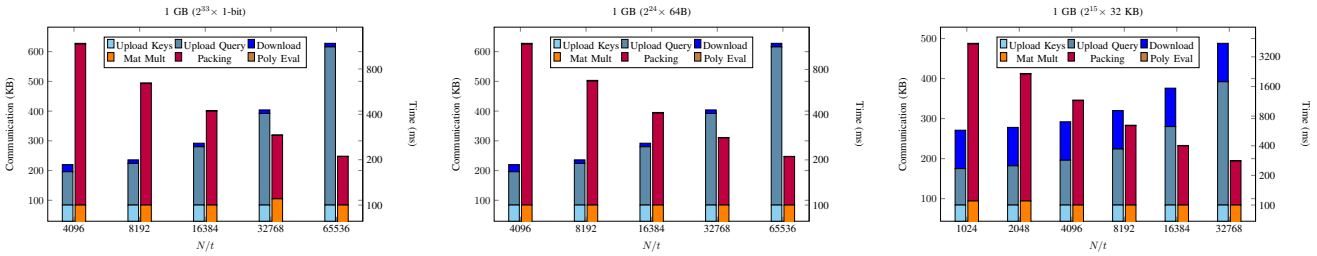

Figure 2: Breakdown of Communication and Online Time of InsPIRe over 1 GB database for various entry sizes as a function of the first dimension (N/t). In each case, we use the largest polynomial degree that satisfies correctness.

in a higher runtime. For large enough interpolation degree, the most costly step in the protocol is the packing, which is compatible with the analysis in Theorem 5.

For the communication costs, the key size and the response size are fixed, regardless of the interpolation degree. The query consists of two parts, the encrypted indicator vector and the RGSW encrypted evaluation point. The size of the RGSW ciphertext is fixed (84 KB), regardless of the interpolation degree. The length of the encrypted indicator vector depends on the first dimension of the database, which inversely depends on the size of each row, which is linear in the interpolation degree. Hence, for larger interpolation degree, the query size reduces, which in turn reduces the overall communication cost.

PIR Evaluation with Various Entry Size. Table 2 shows our evaluation for 1-bit, 64B, and 32KB entries respectively. We compare with the most competitive protocols that support the specified entry size.

Our evaluation shows that for all configurations, our proposed constructions outperform existing work. For 1-bit entries, InsPIRe$_0$ is strictly better than related work, both in terms of communication cost and server runtime. While Table 2 only shows one data point for InsPIRe that outperforms both in communication and server runtime, we note that InsPIRe can achieve even lower communication costs at the cost of higher runtime as shown in Table 2.

The breakdown of the communication cost shows that the cryptographic key material of InsPIRe$_0$ and InsPIRe is over 4x smaller than all other protocols. This is due to the smaller keys used in InspiRING. Similarly, the query and the response size of InsPIRe are smaller than that of all other protocols. This is mainly due to our homomorphic polynomial evaluation technique that allows us to efficiently reduce the PIR response using small additional query overhead.

In summary, InsPIRe strictly improves over existing PIR schemes with server-side preprocessing in the CRS model, and may also be parameterized to optimize specific metrics such as communication.

6. Applications of InsPIRe

The server-side preprocessing and low communication of InsPIRe makes it an ideal candidate for the applications of PIR where an apriori setup is not feasible. We explore two such applications and show how InsPIRe is suitable solution with very good performance. The first application is the use of PIR in IPFS [59] for private queries and the second is private device enrollment in Chrome OS [79]. We use the same experimental setup as Section 5 of single-threaded execution on an Intel Xeon CPU @ 2.6 GHz.

6.1. Private Queries in IPFS

IPFS is a distributed file system which gives clients the ability to access content by querying multiple peers in the network to acquire the necessary routing information and eventually, the content itself. The process of locating and retrieving content can be simplified to three types of database queries, i.e., peer routing, content discovery, and content retrieval. Mazmudar et al. [59] showed how these queries can be done privately with PIR. However, existing PIR protocols require exchanging large key material or a setup phase, which are not compatible with the query pattern in IPFS. A user iteratively contacts different servers to access different parts of the routing information, so the high cost of setup is not amortized over many queries. Hence, the authors proposed alternative PIR protocols which required no setup and had low overall communication costs.

InsPIRe can be used as a more suitable solution for this application, given the server-side preprocessing and extremely low communication. Table 3 shows examples of the communication and computation costs of using PIR in IPFS. We provide the communication and computation cost of the best solution for each functionality of Mazmudar et al. [59] and show the corresponding cost of using InsPIRe. In the case of Content Discovery and Content Retrieval, we use InsPIRe with the parameters specified in Section 5 but in Peer Routing, due to the extremely small database size, we use smaller parameters ($d = 1024$) to achieve very low communication cost. InsPIRe provides strict improvement in communication and computation for all three functionalities.

6.2. Privacy-Preserving Device Enrollment

Upon starting a Chrome OS device, the enrollment process requires checking the membership of the device in a server-held database. Since the introduction of Chrome 94, Google has performed this membership check in a privacy-preserving manner using techniques such as PIR [79]. As Chrome devices perform the device enrollment procedure

TABLE 2: PIR performance metrics for three database sizes (1 GB, 8 GB, and 32 GB) and three entry sizes (1-bit, 64B, and 32 KB). Highlighted values indicate the best option for each metric in each category.

Metric	1-bit entries				64 B entries			32 KB entries			
	HintlessPIR	YPIR	InsPIRe$_0$	InsPIRe	HintlessPIR	YPIR	Ours	KSPIR	HintlessPIR	YPIR	Ours
1 GB Database											
	($2^{33}\times$ 1-bit)				($2^{24}\times$ 64B)			($2^{15}\times$ 32 KB)			
Offline Time	213 s	32 s	43 s	99 s	213 s	119 s	98 s	14 s	213 s	119 s	86 s
Upload (Keys)	360 KB	462 KB	84 KB	84 KB	360 KB	462 KB	84 KB	2352 KB	360 KB	462 KB	84 KB
Upload (Query)	128 KB	384 KB	384 KB	308 KB	128 KB	112 KB	308 KB	14 KB	128 KB	112 KB	308 KB
Download	1748 KB	12 KB	36 KB	12 KB	1748 KB	228 KB	12 KB	224 KB	1748 KB	228 KB	96 KB
Total Comm.	2236 KB	858 KB	504 KB	404 KB	2236 KB	802 KB	404 KB	2590 KB	2236 KB	802 KB	488 KB
Server Time	750 ms	140 ms	120 ms	280 ms	750 ms	600 ms	280 ms	780 ms	750 ms	600 ms	280 ms
Throughput	1370 MB/s	7420 MB/s	8750 MB/s	3670 MB/s	1370 MB/s	1720 MB/s	3670 MB/s	1310 MB/s	1370 MB/s	1720 MB/s	3620 MB/s
8 GB Database											
	($2^{36}\times$ 1-bit)				($2^{27}\times$ 64B)			($2^{18}\times$ 32 KB)			
Offline Time	1500 s	187 s	200 s	581 s	1500 s	283 s	581 s	118 s	1500 s	283 s	597 s
Upload (Keys)	360 KB	462 KB	84 KB	84 KB	360 KB	462 KB	84 KB	2688 KB	360 KB	462 KB	84 KB
Upload (Query)	512 KB	1024 KB	1024 KB	532 KB	512 KB	448 KB	532 KB	14 KB	512 KB	448 KB	532 KB
Download	3316 KB	12 KB	36 KB	12 KB	3316 KB	444 KB	12 KB	224 KB	3316 KB	444 KB	96 KB
Total Comm.	4188 KB	1498 KB	1144 KB	628 KB	4188 KB	1354 KB	628 KB	2926 KB	4188 KB	1354 KB	712 KB
Server Time	2030 ms	830 ms	810 ms	1360 ms	2030 ms	1850 ms	1360 ms	5910 ms	2030 ms	1850 ms	1340 ms
Throughput	4040 MB/s	9830 MB/s	10060 MB/s	6010 MB/s	4040 MB/s	4420 MB/s	6010 MB/s	1390 MB/s	4040 MB/s	4420 MB/s	6110 MB/s
32 GB Database											
	($2^{38}\times$ 1-bit)				($2^{29}\times$ 64B)			($2^{20}\times$ 32 KB)			
Offline Time	5700 s	724 s	731 s	2306 s	5700 s	706 s	2306 s	446 s	5700 s	706 s	2306 s
Upload (Keys)	360 KB	462 KB	84 KB	84 KB	360 KB	462 KB	84 KB	2912 KB	360 KB	462 KB	84 KB
Upload (Query)	1024 KB	2048 KB	2048 KB	980 KB	1024 KB	896 KB	980 KB	14 KB	1024 KB	896 KB	980 KB
Download	6452 KB	12 KB	36 KB	96 KB	6452 KB	888 KB	96 KB	224 KB	6452 KB	888 KB	96 KB
Total Comm.	7836 KB	2522 KB	2168 KB	1160 KB	7836 KB	2246 KB	1160 KB	3150 KB	7836 KB	2246 KB	1160 KB
Server Time	5910 ms	3390 ms	3390 ms	4320 ms	5910 ms	5610 ms	4320 ms	50570 ms	5910 ms	5610 ms	4320 ms
Throughput	5550 MB/s	9680 MB/s	9670 MB/s	7580 MB/s	5550 MB/s	5840 MB/s	7580 MB/s	650 MB/s	5550 MB/s	5840 MB/s	7580 MB/s

TABLE 3: Costs of PIR queries in the three functionalities of IPFS. *Comm.* denotes the total communication required to serve the query and *Comp.* denotes the server runtime.

Step	Metric	Current [59]	InsPIRe	Improvement
Peer Routing (256×1.5 KB)	Comm.	$>$ 100 KB	69 KB	32%
	Comp.	$>$ 56 ms	51 ms	8%
Content Discovery (200k Records)	Comm.	$>$ 280 KB	128 KB	46%
	Comp.	$>$ 875 ms	123 ms	86%
Content Retrieval ($2^{14}\times$ 256 KB)	Comm.	$>$ 2.1 MB	1.02 MB	51%
	Comp.	$>$ 3.0 s	1.5 s	50%

with PIR immediately when it is powered on for the first time or right after a factory reset, there is no opportunity for the device to perform setup with the server. InsPIRe is an ideal solution for this application due to the server-side preprocessing step. While the precise number of Chromebooks is not disclosed, publicly available data shows over 20 million devices are sold every year from 2019 until 2023[5]. We assume each device requires about 64 bytes of information from the database and provide runtimes for various number of devices. The cost of device enrollment using InsPIRe is shown in Table 4.

Table 4 shows that the concrete cost of using PIR for device enrollment is only a few hundred KiloBytes, which is very practical network cost for a personal computer.

5. https://www.statista.com/statistics/749890/worldwide-chromebook-unit-shipments/

TABLE 4: Cost of PIR for Chrome Device Enrollment

Devices	DB Size	Offline Time	Communication	Response Time
20 M	1.19 GB	78 s	292 KB	416 ms
40 M	2.38 GB	131 s	292 KB	815 ms
80 M	4.76 GB	241 s	304 KB	1400 ms

7. Related Works

PIR was introduced by Chor *et al.* [23] in the multi-server setting and Kushilevitz and Ostrovsky [47] in the single-server setting. Most early single-server PIR schemes were built upon number-theoretic assumptions including [70], [28], [69] using a linearly homomorphic encryption scheme. Multi-server PIR has been built without any cryptographic assumptions [34] and one-way functions [11].

PIR using Server-Stored Client-Specific Keys. In recent years, many works aim to design practically efficient single-server PIR schemes using lattice-based homomorphic encryption starting with XPIR [60] leading to a long line of work including [4], [3], [66], [1], [61], [16]. For efficiency purposes, these protocols assume that the server stores a key that is specific to each client that performs PIR queries. With these server-stored client-specific keys, these schemes obtain very low query communication, but require large computation due to relying on RLWE ciphertexts.

PIR using Client-Stored Database Hints. In another line of work, prior works consider single-server PIR where clients download and store database hints that will be used later for queries. The works of SimplePIR [42] and FrodoPIR [29]

built schemes using LWE, but require each client to download large database hints before querying. In an orthogonal line, it was shown that client-stored database hints could enable sublinear query times [71], [27], [46], [26], [43]. Several very recent works have considered practical single-server implementations of this PIR paradigm such as [80], [51], [75], [36]. However, these works require either streaming the entire database in the offline phase or requiring the server to perform heavy computation per client.

PIR using the CRS Model. Very recently, several works have considered PIR without offline communication to avoid the obstacles and costs of offline preprocessing. Starting with Tiptoe [41], follow-ups such as HintlessPIR [52] and YPIR [62] consider practical PIR in the CRS model. Prior to InsPIRe, all these works sacrificed larger query communication in order to eliminate offline communication. In a more theoretical line of work, a recent breakthrough work [55] presented a doubly efficient PIR construction with sublinear query time and silent preprocessing from RLWE, but the practical costs remain impractical [68].

PIR with Additional Features. PIR with more advanced query functionalities have also been studied such as batch PIR that efficiently retrieves multiple entries at once. Some examples include [56], [40], [5], [4], [78], [67], [8]. Constructions for keyword PIR has also been presented where queries are performed over sparse databases consisting of keys and values (see [3], [58], [72], [17] as examples). PIR with additional security features has also been studied such as in the presence of malicious servers including [24], [30] as well as symmetric PIR with database privacy [3], [54].

Ring Packing. The problem of ring packing to transform LWE to RLWE ciphertexts has been studied in three different classes. The row method [18] uses the least amount of cryptographic material while the diagonal [44] and column methods [21], [64], [10], [7] use more cryptographic material to obtain faster packing times.

Labeled Unbalanced PSI. The idea of using polynomial interpolation to represent sets and homomorphic polynomial evaluation for retrieval have been explored in the context of labeled unbalanced PSI [20], [19], [25], but the detailed constructions are quite different from ours. In particular, in these works, the interpolation is done over the plaintext SIMD slots (e.g. over $\mathbb{Z}_p$), while our polynomial interpolation is done over the *entire plaintext ring* R_p. Importantly, the PSI constructions encode the evaluation points as SIMD slots which are then encrypted as RLWE ciphertexts. The encrypted plaintexts generally have high norms, and so each homomorphic multiplication results in a large multiplicative noise growth. To overcome the large depth homomorphic multiplications, these works proposed tricks such as sending multiple powers of the evaluation points to the sender. In contrast, our homomorphic evaluation uses RGSW encrypted unit monomials which only incur additive noise growth for each multiplication, allowing the use of efficient RLWE parameters.

8. Conclusions

We present InsPIRe that obtains higher throughput and smaller query communication than all prior PIR schemes, using only server-side preprocessing. Along the way, we introduce a novel ring packing algorithm, InspiRING, requiring smaller cryptographic material and faster online packing times as well as a new approach to PIR using homomorphic polynomial evaluation.

References

[1] Ishtiyaque Ahmad, Yuntian Yang, Divyakant Agrawal, Amr El Abbadi, and Trinabh Gupta. Addra: Metadata-private voice communication over fully untrusted infrastructure. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 313–329. USENIX Association, July 2021.

[2] Martin R Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.

[3] Asra Ali, Tancrède Lepoint, Sarvar Patel, Mariana Raykova, Phillipp Schoppmann, Karn Seth, and Kevin Yeo. Communication-computation trade-offs in PIR. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 1811–1828. USENIX Association, August 2021.

[4] Sebastian Angel, Hao Chen, Kim Laine, and Srinath Setty. PIR with Compressed Queries and Amortized Query Processing. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 962–979, San Francisco, CA, May 2018. IEEE.

[5] Sebastian Angel and Srinath Setty. Unobservable communication over fully untrusted infrastructure. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 551–569, 2016.

[6] Apple. Getting up-to-date calling and blocking information for your app. https://developer.apple.com/documentation/sms_and_call_reporting/getting_up-to-date_calling_and_blocking_information_for_your_app, 2024.

[7] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, Jai Hyun Park, and Damien Stehlé. HERMES: Efficient ring packing using MLWE ciphertexts and application to transciphering. In *CRYPTO 2023, Part IV*, LNCS, pages 37–69, August 2023.

[8] Alexander Bienstock, Sarvar Patel, Joon Young Seo, and Kevin Yeo. Batch PIR and labeled PSI with oblivious ciphertext compression. USENIX Association, 2024.

[9] Nikita Borisov, George Danezis, and Ian Goldberg. DP5: A private presence service. *PoPETs*, 2015(2):4–24, April 2015.

[10] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. Chimera: Combining ring-lwe-based fully homomorphic encryption schemes. *Journal of Mathematical Cryptology*, 14(1):316–338, 2020.

[11] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1292–1303. ACM Press, October 2016.

[12] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical gapsvp. In *Annual cryptology conference*, pages 868–886. Springer, 2012.

[13] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)*, 6(3):1–36, 2014.

[14] Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) lwe. *SIAM Journal on computing*, 43(2):831–871, 2014.

[15] D. J. Britten and F. W. Lemire. A structure theorem for rings supporting a discrete fourier transform. *SIAM Journal on Applied Mathematics*, 41(2):222–226, 1981.

[16] Alexander Burton, Samir Jordan Menon, and David J Wu. Respire: High-rate pir for databases with small records. In *ACM CCS*, 2024.

[17] Sofía Celi and Alex Davidson. Call me by my name: Simple, practical private information retrieval for keyword queries. In *ACM CCS*, 2024.

[18] Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. Efficient Homomorphic Conversion Between (Ring) LWE Ciphertexts. In Kazue Sako and Nils Ole Tippenhauer, editors, *Applied Cryptography and Network Security*, volume 12726, pages 460–479. Springer International Publishing, Cham, 2021.

[19] Hao Chen, Zhicong Huang, Kim Laine, and Peter Rindal. Labeled psi from fully homomorphic encryption with malicious security. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1223–1237, 2018.

[20] Hao Chen, Kim Laine, and Peter Rindal. Fast private set intersection from homomorphic encryption. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1243–1255, 2017.

[21] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster packed homomorphic operations and efficient circuit bootstrapping for TFHE. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 377–408, December 2017.

[22] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Tfhe: fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

[23] Benny Chor, Eyal Kushilevitz, Oded Goldreich, and Madhu Sudan. Private information retrieval. *Journal of the ACM*, 45(6):965–981, November 1998.

[24] Simone Colombo, Kirill Nikitin, Henry Corrigan-Gibbs, David J. Wu, and Bryan Ford. Authenticated private information retrieval. pages 3835–3851. USENIX Association, 2023.

[25] Kelong Cong, Radames Cruz Moreno, Mariana Botelho da Gama, Wei Dai, Ilia Iliashenko, Kim Laine, and Michael Rosenberg. Labeled psi from homomorphic encryption with reduced computation and communication. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 1135–1150, 2021.

[26] Henry Corrigan-Gibbs, Alexandra Henzinger, and Dmitry Kogan. Single-server private information retrieval with sublinear amortized time. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 3–33, May / June 2022.

[27] Henry Corrigan-Gibbs and Dmitry Kogan. Private information retrieval with sublinear online time. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 44–75, May 2020.

[28] Ivan Damgård and Mats Jurik. A generalisation, a simplification and some applications of Paillier's probabilistic public-key system. In Kwangjo Kim, editor, *PKC 2001*, volume 1992 of *LNCS*, pages 119–136, February 2001.

[29] Alex Davidson, Gonçalo Pestana, and Sofía Celi. FrodoPIR: Simple, Scalable, Single-Server Private Information Retrieval. *Proceedings on Privacy Enhancing Technologies*, 2023(1):365–383, January 2023.

[30] Leo de Castro and Keewoo Lee. Verisimplepir: verifiability in simplepir at no online cost for honest servers. In *USENIX Security*, 2024.

[31] Leo de Castro, Kevin Lewi, and Edward Suh. WhisPIR: Stateless private information retrieval with low communication. Cryptology ePrint Archive, Paper 2024/266, 2024.

[32] Daniel Demmler, Peter Rindal, Mike Rosulek, and Ni Trieu. PIR-PSI: Scaling private contact discovery. *PoPETs*, 2018(4):159–178, October 2018.

[33] David Steven Dummit, Richard M Foote, et al. *Abstract algebra*, volume 3. Wiley Hoboken, 2004.

[34] Zeev Dvir and Sivakanth Gopi. 2-server PIR with sub-polynomial communication. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *47th ACM STOC*, pages 577–584. ACM Press, June 2015.

[35] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, 2012.

[36] Ben Fisch, Arthur Lazzaretti, Zeyu Liu, and Charalampos Papamanthou. ThorPIR: Single server PIR via homomorphic thorp shuffles. In *ACM CCS*, 2024.

[37] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pages 169–178, 2009.

[38] Matthew Green, Watson Ladd, and Ian Miers. A protocol for privately reporting ad impressions at scale. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1591–1601. ACM Press, October 2016.

[39] Trinabh Gupta, Natacha Crooks, Whitney Mulhern, Srinath Setty, Lorenzo Alvisi, and Michael Walfish. Scalable and private media consumption with popcorn. In *13th USENIX symposium on networked systems design and implementation (NSDI 16)*, pages 91–107, 2016.

[40] Ryan Henry. Polynomial batch codes for efficient IT-PIR. *PoPETs*, 2016(4):202–218, October 2016.

[41] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. Private web search with tiptoe. In *Proceedings of the 29th symposium on operating systems principles*, pages 396–416, 2023.

[42] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast Single-Server private information retrieval. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3889–3905, Anaheim, CA, August 2023. USENIX Association.

[43] Alexander Hoover, Sarvar Patel, Giuseppe Persiano, and Kevin Yeo. Plinko: Single-server PIR with efficient updates via invertible PRFs. Cryptology ePrint Archive, Report 2024/318, 2024.

[44] Wen jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. PEGASUS: Bridging polynomial and non-polynomial evaluations in homomorphic encryption. In *2021 IEEE Symposium on Security and Privacy*, pages 1057–1073. IEEE Computer Society Press, May 2021.

[45] Daniel Kales, Christian Rechberger, Thomas Schneider, Matthias Senker, and Christian Weinert. Mobile private contact discovery at scale. In Nadia Heninger and Patrick Traynor, editors, *USENIX Security 2019*, pages 1447–1464. USENIX Association, August 2019.

[46] Dmitry Kogan and Henry Corrigan-Gibbs. Private blocklist lookups with checklist. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 875–892. USENIX Association, August 2021.

[47] Eyal Kushilevitz and Rafail Ostrovsky. Replication is NOT needed: SINGLE database, computationally-private information retrieval. In *38th FOCS*, pages 364–373. IEEE Computer Society Press, October 1997.

[48] Albert Kwon, David Lazar, Srinivas Devadas, and Bryan Ford. Riffle: An efficient communication system with strong anonymity. *PoPETs*, 2016(2):115–134, April 2016.

[49] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.

[50] Kristin Lauter, Sreekanth Kannepalli, Kim Laine, and Radames Cruz Moreno. Password Monitor: Safeguarding passwords in Microsoft Edge. https://www.microsoft.com/en-us/research/blog/password-monitor-safeguarding-passwords-in-microsoft-edge/, 2021.

[51] Arthur Lazzaretti and Charalampos Papamanthou. Single pass client-preprocessing private information retrieval. USENIX Association, 2024.

[52] Baiyu Li, Daniele Micciancio, Mariana Raykova, and Mark Schultz. Hintless single-server private information retrieval. LNCS, pages 183–217, August 2024.

[53] Lucy Li, Bijeeta Pal, Junade Ali, Nick Sullivan, Rahul Chatterjee, and Thomas Ristenpart. Protocols for checking compromised credentials. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 1387–1403. ACM Press, November 2019.

[54] Chengyu Lin, Zeyu Liu, and Tal Malkin. XSPIR: Efficient symmetrically private information retrieval from ring-LWE. In Vijayalakshmi Atluri, Roberto Di Pietro, Christian Damsgaard Jensen, and Weizhi Meng, editors, *ESORICS 2022, Part I*, volume 13554 of *LNCS*, pages 217–236, September 2022.

[55] Wei-Kai Lin, Ethan Mook, and Daniel Wichs. Doubly efficient private information retrieval and fully homomorphic RAM computation from ring LWE. pages 595–608. ACM Press, 2023.

[56] Wouter Lueks and Ian Goldberg. Sublinear scaling for multi-client private information retrieval. In Rainer Böhme and Tatsuaki Okamoto, editors, *FC 2015*, volume 8975 of *LNCS*, pages 168–186, January 2015.

[57] Ming Luo, Feng-Hao Liu, and Han Wang. Faster fhe-based single-server private information retrieval. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 1405–1419, 2024.

[58] Rasoul Akhavan Mahdavi and Florian Kerschbaum. Constant-weight PIR: Single-round keyword PIR via constant-weight equality operators. pages 1723–1740. USENIX Association, 2022.

[59] Miti Mazmudar, Shannon Veitch, and Rasoul Akhavan Mahdavi. Peer2PIR: Private Queries for IPFS . In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 4438–4456, San Fransisco, CA, USA, May 2025. IEEE Computer Society.

[60] Carlos Aguilar Melchor, Joris Barrier, Laurent Fousse, and Marc-Olivier Killijian. Xpir: Private information retrieval for everyone. *Proceedings on Privacy Enhancing Technologies*, pages 155–174, 2016.

[61] Samir Jordan Menon and David J. Wu. SPIRAL: Fast, High-Rate Single-Server PIR via FHE Composition. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 930–947, San Francisco, CA, USA, May 2022. IEEE.

[62] Samir Jordan Menon and David J. Wu. YPIR: High-throughput single-server PIR with silent preprocessing. USENIX Association, 2024.

[63] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 700–718, April 2012.

[64] Daniele Micciancio and Jessica Sorrell. Ring packing and amortized FHEW bootstrapping. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *ICALP 2018*, volume 107 of *LIPIcs*, pages 100:1–100:14. Schloss Dagstuhl, July 2018.

[65] Prateek Mittal, Femi Olumofin, Carmela Troncoso, Nikita Borisov, and Ian Goldberg. Pir-tor: scalable anonymous communication using private information retrieval. In *Proceedings of the 20th USENIX Conference on Security*, SEC'11, page 31, USA, 2011. USENIX Association.

[66] Muhammad Haris Mughees, Hao Chen, and Ling Ren. OnionPIR: Response efficient single-server PIR. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2292–2306. ACM Press, November 2021.

[67] Muhammad Haris Mughees and Ling Ren. Vectorized batch private information retrieval. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 437–452, 2023.

[68] Hiroki Okada, Rachel Player, Simon Pohmann, and Christian Weinert. Towards practical doubly-efficient private information retrieval. In *Financial Cryptography and Data Security 2024 (FC '24)*, 2024.

[69] Rafail Ostrovsky and William E. Skeith, III. A survey of single-database private information retrieval: Techniques and applications (invited talk). In Tatsuaki Okamoto and Xiaoyun Wang, editors, *PKC 2007*, volume 4450 of *LNCS*, pages 393–411, April 2007.

[70] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *EUROCRYPT'99*, volume 1592 of *LNCS*, pages 223–238, May 1999.

[71] Sarvar Patel, Giuseppe Persiano, and Kevin Yeo. Private stateful information retrieval. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 1002–1019. ACM Press, October 2018.

[72] Sarvar Patel, Joon Young Seo, and Kevin Yeo. Don't be dense: Efficient keyword PIR for sparse databases. pages 3853–3870. USENIX Association, 2023.

[73] Chris Peikert et al. A decade of lattice cryptography. *Foundations and trends® in theoretical computer science*, 10(4):283–424, 2016.

[74] Chris Peikert and Brent Waters. Lossy trapdoor functions and their applications. In Richard E. Ladner and Cynthia Dwork, editors, *40th ACM STOC*, pages 187–196. ACM Press, May 2008.

[75] Ling Ren, Muhammad Haris Mughees, and I Sun. Simple and practical amortized sublinear private information retrieval. In *ACM CCS*, 2024.

[76] Kurt Thomas, Jennifer Pullman, Kevin Yeo, Ananth Raghunathan, Patrick Gage Kelley, Luca Invernizzi, Borbala Benko, Tadek Pietraszek, Sarvar Patel, Dan Boneh, and Elie Bursztein. Protecting accounts from credential stuffing with password breach alerting. In Nadia Heninger and Patrick Traynor, editors, *USENIX Security 2019*, pages 1556–1571. USENIX Association, August 2019.

[77] Frank Wang, Catherine Yun, Shafi Goldwasser, Vinod Vaikuntanathan, and Matei Zaharia. Splinter: Practical private queries on public data. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 299–313, 2017.

[78] Kevin Yeo. Cuckoo hashing in cryptography: Optimal parameters, robustness and applications. In *CRYPTO 2023, Part IV*, LNCS, pages 197–230, August 2023.

[79] Kevin Yeo and Sarvar Patel. Protecting your device information with private set membership. https://security.googleblog.com/2021/10/protecting-your-device-information-with.html, 2021.

[80] Mingxun Zhou, Andrew Park, Wenting Zheng, and Elaine Shi. Piano: Extremely Simple, Single-Server PIR with Sublinear Server Computation . In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4296–4314. IEEE Computer Society, 2024.

Appendix A. Supplementary Material for PIR

A.1. Security and Correctness Definitions of PIR

Security. We define single query privacy for a PIR scheme using a standard indistinguishability game.

1) **Setup Phase:**
   a) On input security parameter 1^λ, $\mathcal{A}$ outputs a database $\mathbf{D}$.
   b) The challenger computes $(\mathsf{pp}, \mathbf{D}') \leftarrow \mathsf{Setup}(1^\lambda, \mathbf{D})$ and gives it to $\mathcal{A}$.
2) **Challenge Phase:**
   a) $\mathcal{A}$ outputs a pair of challenge indices $(\mathsf{idx}^{(0)}, \mathsf{idx}^{(1)})$.

b) The challenger chooses a bit $b \in \{0,1\}$ uniformly at random.
c) The challenger computes the challenge query $(\mathsf{st}^{(b)}, \mathsf{qry}^{(b)}) \leftarrow \mathsf{Query}(\mathsf{pp}, \mathsf{idx}^{(b)})$. The challenger sends $\mathsf{qry}^{(b)}$ to $\mathcal{A}$.

3) **Guess Phase:** $\mathcal{A}$ outputs a bit b'.

The advantage of the adversary $\mathcal{A}$ in this game is:

$$\mathsf{Adv}^{\mathsf{QP}}_{\mathcal{A}}(\lambda) = |\Pr[b' = b] - (1/2)|$$

We say that a PIR scheme achieves single query privacy if for any PPT adversary $\mathcal{A}$, its advantage $\mathsf{Adv}^{\mathsf{QP}}_{\mathcal{A}}(\lambda)$ is a negligible function in λ. As in SimplePIR [42], we may extend the notion of single query privacy to a sequence of Q queries. We defer the details to the full paper.

Correctness. For all $\lambda \in \mathbb{N}$, all databases $\mathbf{D}$, and all indices idx, let $(\mathsf{pp}, \mathbf{D}') \leftarrow \mathsf{Setup}(1^\lambda, \mathbf{D})$, $(\mathsf{st}, \mathsf{qry}) \leftarrow \mathsf{Query}(\mathsf{pp}, \mathsf{idx})$ and $\mathsf{resp} \leftarrow \mathsf{Respond}(\mathsf{pp}, \mathbf{D}', \mathsf{qry})$. For a correctness parameter δ, we say that a PIR protocol is $(1-\delta)$-correct if $\Pr[\mathsf{Extract}(\mathsf{pp}, \mathsf{st}, \mathsf{resp}) = \mathbf{D}[\mathsf{idx}]] \geq 1-\delta$.

A.2. Security Proof of InsPIRe

InsPIRe relies on the *Key-Dependent RLWE Hardness* assumption that additionally provides an oracle to the encryptions of scaled automorphic images of the secret key. We refer readers to Definition A.1 in YPIR [62] for details.

Theorem 7. *Let λ be the security parameter, $z_{ks}, \ell_{ks}, z_{gsw}, \ell_{gsw}$ be the gadget parameters for* InspiRING *and the RGSW encryption respectively. Suppose that the key-dependent RLWE hardness assumption holds for number of samples $m \geq N/t + 2\ell_{ks} + 2\ell_{gsw}$. Then* InsPIRe *satisfies query privacy.*

We sketch the proof below. The proof uses standard reduction techniques to create an algorithm $\mathcal{B}$ that can break the key-dependent RLWE hardness problem using an adversary $\mathcal{A}$ that can break InsPIRe. The general idea is that, using multiple RLWE samples and oracle queries, $\mathcal{B}$ can perfectly simulate the view of $\mathcal{A}$ in the indistinguishability game. This is because $\mathcal{B}$ can essentially generate queries (LWE and RGSW ciphertext) using RLWE samples and generate the two key-switching matrices from oracle queries. We note that RLWE samples can be used to generate LWE encryptions by interpreting the RLWE samples as LWE samples. We leave the formal proof to the full paper.

Appendix B. Supplementary Material for Ring Packing

B.1. LWE to Intermediate Ciphertexts

We give a detailed mathematical derivation of the first stage of our packing algorithm that transforms the input LWE ciphertexts into intermediate ciphertexts. As discussed, our approach transforms the RLWE ciphertext $(\tilde{a}, \tilde{b})$ into an intermediate ciphertext that resembles an MLWE ciphertext. A key characteristic of this transformed ciphertext is that its non-constant term coefficients are zeroed out. Furthermore, these intermediate ciphertexts retain homomorphic properties, enabling aggregation of multiple ciphertexts into a single one that contains the LWE messages as the coefficients.

To facilitate this transformation, we introduce an alternative interpretation of the $\tilde{b}$ component, leveraging Lemma 1. In essence, Lemma 1 states that for any polynomial $p(X) = \sum_{i=0}^{d-1} c_i X^i$, summing $d/2$ versions of $\tau_g^j(p) + \tau_h \circ \tau_g^j(p)$ (obtained by applying the automorphism $X \mapsto X^{g^j}$ and $X \mapsto X^{g^j \cdot h}$ for $j \in \{0, \ldots, d/2-1\}$) yields a polynomial where all coefficients are zero except the constant term, which is scaled by a factor of d. As we will show, this operation is crucial for isolating the LWE message m into a constant term polynomial.

Our initial step involves applying the operator Tr from Lemma 1 to the $\tilde{b}$ component of the RLWE representation $(\tilde{a}, \tilde{b})$. More specifically, from Eq. 1, we may re-interpret everything over $\mathbb{Z}[X]/(X^d+1)$ and obtain $\tilde{b} = -\tilde{a}\tilde{s} + \tilde{m} + q\tilde{u}$ for some $\tilde{u} \in \mathbb{Z}[X]/(X^d+1)$. Applying Tr to both sides, we get:

$$\begin{aligned}\mathrm{Tr}(\tilde{b}) &= \mathrm{Tr}(-\tilde{a}\tilde{s} + \tilde{m} + q\tilde{u}) \\ &= -\mathrm{Tr}(\tilde{a}\tilde{b}) + \mathrm{Tr}(\tilde{m}) + q \cdot \mathrm{Tr}(\tilde{u}).\end{aligned}$$

Given that $\tilde{b}$ is a constant polynomial, $\mathrm{Tr}(\tilde{b}) = \sum_{j=0}^{d/2-1} \tau_g^j(\tilde{b}) + \tau_h \circ \tau_g^j(\tilde{b}) = d \cdot \tilde{b}$. Assuming d is invertible mod q (which is true when q is odd), we can write:

$$\begin{aligned}\tilde{b} &= d^{-1}\left(-\mathrm{Tr}(\tilde{a}\tilde{s}) + \mathrm{Tr}(\tilde{m})\right) \pmod q \\ &= -d^{-1}\mathrm{Tr}(\tilde{a}\tilde{s}) + d^{-1}\mathrm{Tr}(\tilde{m}) \pmod q.\end{aligned}$$

We have $\mathrm{Tr}(\tilde{m}) = d \cdot m$. We define a new message representation as $\hat{m}(X) := d^{-1} \cdot \mathrm{Tr}(\tilde{m}) = m$.

Substituting this expression for $\tilde{b}$:

$$\begin{aligned}\tilde{b} &= -d^{-1} \cdot \mathrm{Tr}(\tilde{a}\tilde{b}) + \hat{m} \pmod q \\ &= -\left(\sum_{j=0}^{d/2-1} \left(d^{-1} \cdot \tau_g^j(\tilde{a})\right) \cdot \tau_g^j(\tilde{s})\right) \\ &\quad - \left(\sum_{j=0}^{d/2-1} \left(d^{-1} \cdot \tau_h \circ \tau_g^j(\tilde{a})\right) \cdot \left(\tau_h \circ \tau_g^j(\tilde{s})\right)\right) + \hat{m} \pmod q \\ &= -\langle \hat{\mathbf{a}}, \hat{\mathbf{s}} \rangle + \hat{m} \pmod q\end{aligned}$$

The second equality follows from the definition of Tr and the property that τ_g and τ_h are ring automorphisms. The final equality is written succinctly as an inner product of two vectors $\hat{\mathbf{a}}, \hat{\mathbf{s}} \in R_q^d$. Here, $\hat{\mathbf{a}}[j] := d^{-1} \cdot \tau_g^j(\tilde{a})$ and $\hat{\mathbf{a}}[j + d/2] := d^{-1} \cdot \tau_h \circ \tau_g^j(\tilde{a})$ for $j < d/2$ e.g. the second half is an automorphic image of the first half by τ_h. Similarly, $\hat{\mathbf{s}}[j] := \tau_g^j(\tilde{s})$ and $\hat{\mathbf{s}}[j + d/2] := \tau_h \circ \tau_g(\tilde{s})$ for $j < d/2$.

This formulation allows $\tilde{b}$ to be interpreted as a pseudorandom component of an MLWE-like ciphertext with the random component $\hat{\mathbf{a}}$ and the secret key $\hat{\mathbf{s}}$.

B.2. Conversion to RLWE Ciphertext

This section provides the detailed procedure for converting the aggregated intermediate ciphertext $\mathsf{IRCtx}(\hat{m}) = (\hat{\mathbf{a}}_{agg}, \tilde{b}_{agg}) \in R_q^d \times R_q$ into a standard two-component RLWE ciphertext. The input $\mathsf{IRCtx}(\hat{m}_{agg})$ encrypts the packed message $\hat{m}_{agg}$ (as defined in the main body) and consists of $d+1$ ring elements. The components $\hat{\mathbf{a}}_{\mathbf{agg}}[j]$ are associated with secret key shares $\hat{\mathbf{s}}[j] = \tau_g^j(\tilde{s})$ and $\hat{\mathbf{s}}[j+d/2] = \tau_h \circ \tau_g^j(\tilde{s})$ for $j < d/2$. The goal is to obtain a ciphertext (a_{fin}, b_{fin}) encrypted under base secret key $\tilde{s}$.

The conversion relies on iteratively reducing the number of components using key-switching.

Key-Switching Preliminaries. Recall the key-switching key generation algorithm $\mathsf{KS.Setup}(s_{in}, s_{out})$ which produces a key $\mathbf{K}$ that can transform a ciphertext component encrypted under s_{in} into one encrypted under s_{out}.

For our specific purpose, we require two elementary key-switching matrices. The first matrix is constructed as $\mathbf{K}_g \leftarrow \mathsf{KS.Setup}(\tau_g(\tilde{s}), \tilde{s})$. By applying Galois automorphisms τ_g^i (resp. $\tau_h \circ \tau_g^i$) to $\mathbf{K}_g$, we obtain a key-switching matrix from $\tau_g^{j+1}(\tilde{s})$ to $\tau_g^j(\tilde{s})$ (resp. $\tau_h \circ \tau_g^{j+1}(\tilde{s})$ to $\tau_h \circ \tau_g^j(\tilde{s})$). The second matrix is constructed as $\mathbf{K}_h \leftarrow \mathsf{KS.Setup}(\tau_h(\tilde{s}), \tilde{s})$ and used in the final step of the algorithm.

The CollapseOne Subroutine. The core building block for this conversion is a subroutine to "collapse" ciphertext components, adapted from standard relinearization techniques. The goal of this subroutine is to use the key-switching to re-express the pseudorandom component masked by one secret key share so that it is now masked by another key share. We refer to these key shares as the source key share and the target key share, respectively.

In line 2, key-switching is performed on the last component $\mathbf{a}[k-1]$ with respect to the current pseudorandom component b. This in turn results in $b' = -\langle \mathbf{a}'', \mathbf{s}' \rangle + m + e_{noise} + e_{ks}$, where $\mathbf{a}''$ and $\mathbf{s}'$ are the "reduced" vectors without the corresponding source key share component. e_{noise} is the initial noise present in the input and e_{ks} is the additive noise from the key-switching. Note that the returned pair $(\mathbf{a}', b')$ is essentially a *partial ciphertext* where $\mathbf{a}'$ masks a portion of the pseudorandom component b'.

The CollapseHalf Subroutine. This subroutine is used to collapse the random components masked by the key shares $\tau_g^k(\tilde{s})$ (resp. $\tau_h \circ \tau_g^k(\tilde{s})$) so that it is masked by a single key share $\tilde{s}$ (resp. $\tau_h(\tilde{s})$).

Using the CollapseOne procedure as the building block, we can iteratively collapse the ciphertext components.

In the k-th iteration of the loop, the component to be eliminated is $\mathbf{a}^{(k)}[k]$, which is associated with the secret $\tau_g^k(\tilde{s})$ (resp. $\tau_h \circ \tau_g^k(\tilde{s})$). The target secret for this component is $\tau_g^{k-1}(\tilde{s})$ (resp. $\tau_h \circ \tau_g^{k-1}(\tilde{s})$). Consequently, the required key-switching matrix is $\mathbf{K}_g^{k-1} = \tau_g^{k-1}(\mathbf{K}_g)$ (resp. $\tau_h \circ \tau_g^{k-1}(\mathbf{K}_g)$ and we can invoke CollapseOne to reduce the number of ciphertext components by one.

After $d/2 - 1$ such key-switching steps, we obtain $(\mathbf{a}^{(0)}, b^{(0)})$. Note, the masking part of $b^{(0)}$ that was originally masked by secret key shares $\tau_g^j(\tilde{s})$ (resp. $\tau_h \circ \tau_g^j(\tilde{s})$) is now masked by $\tilde{s}$ (resp. $\tau_h(\tilde{s})$) with $\mathbf{a}^{(0)}$ as its random component. This process incurs additive noise corresponding to the $d/2 - 1$ key-switchings.

The Collapse Procedure. After invoking CollapseHalf on both halves of the random components, the resultant pseudorandom component b_2 can be expressed as $b_2 = -a_1\tilde{s} - a_2\tau_h(\tilde{s}) + \hat{m}_{agg} + e_{tot}$ where e_{tot} is the total accumulated noise from the $d-2$ key-switchings. The final key-switching using $\mathbf{K}_h$ is performed to eliminate the secret key share $\tau_h(\tilde{s})$ in the expression.

Online Computation. We can see that the online execution of each invocation of $\mathsf{KS.KeySwitch}$ involves an inner product between $\mathbf{y}^{k-1}$ and the gadget decomposed $\mathbf{a}^{(k)}$, which costs $O(\ell)$ polynomial multiplications and additions. This in turn implies that there are in total $O(\ell d)$ polynomial multiplications and additions. In NTT space, each polynomial multiplication and addition costs $O(d)$, and so the total online running time of packing is $O(\ell \cdot d^2)$.

B.3. Benchmarking Ring Packing

In this section, we provide a benchmark of InspiRING compared to existing ring packing algorithms such as CDKS [18] and the packing algorithms used in HintlessPIR [52]. In this benchmark, we pack 2^{12} LWE ciphertexts and measure the offline and online runtime of this procedure. Additionally, we report the size of additional material the client must provide for packing, i.e., the packing keys in the case of InspiRING. We run each protocol using the parameters provided in the respective paper or implementation to achieve the best performance and compare with InsPIRe that is instantiated with similar parameters. Using the notation of Section 3, we use two parameters set for InspiRING which provide 128-bit security based on the lattice estimator:

1) $\log_2 d = 10$, $\log_2 q = 28$, $\log_2 p = 6$, $\ell = 8$, $z = 2^4$
2) $\log_2 d = 11$, $\log_2 q = 56$, $\log_2 p = 15$, $\ell = 3$, $z = 2^{19}$

Table 5 summarizes the results.

TABLE 5: Concrete costs of packing 2^{12} LWE ciphertexts into RLWE ciphertexts using the approach in HintlessPIR [52], CDKS [18] and InspiRING.

Packing Type	HintlessPIR	InspiRING	CDKS	InspiRING
Unpacked Size	16 MB	14 MB	56 MB	56 MB
$\log_2(d, q, p)$	(10, 32, 8)	(10, 28, 6)	(11, 56, 15)	(11, 56, 15)
Key Material	360 KB	**60 KB**	462 KB	**84 KB**
Packed Size	180 KB	**32 KB**	56 KB	**56 KB**
Total Size	540 KB	**92 KB**	518 KB	**140 KB**
Offline Runtime	2.0 s	2.4 s	11 s	36 s
Online Runtime	141 ms	**16 ms**	56 ms	**40 ms**

We make the following observations from this table. Firstly, InspiRING requires significantly smaller key material compared to existing work, specifically, 84%, 76%, and over 99% less key material than CDKS and HintlessPIR, respectively. Secondly, the online time of InspiRING is 28% lower than the fastest existing work, CDKS. This, however,

comes at the cost of a slower offline phase, which can be attributed to the quadratic dependence on d

Lastly, we confirm the analytic noise analysis of InspiRING from Theorem 2. In the same experiment, we observe that the bitlength of $||e_{pack}||_\infty$ for ciphertexts packed using CDKS and InsPIRe ($d = 2048$) are equal to 38.5 and 33.4, respectively. This confirms that proposed ring packing construction has less noise growth than CDKS, up to 5 bits in this example.

B.4. Proof of Lemma 1

First, we state a useful number theoretic fact from [33].

Lemma 3. *[33] Let d be a power of two and $\gamma < d$ also a power of two. Let $g = 2d/\gamma + 1 \in \mathbb{Z}^*_{2d}$. Then $\mathsf{ord}(g) = \gamma$.*

Lemma 4. *Let d be a power of two and $\gamma < d$ also a power of two. Let $g = 2d/\gamma + 1 \in \mathbb{Z}^*_{2d}$. Then the map $g^i \pmod{2d} \mapsto g^i + g - 1 \pmod{2d}$ is a bijection.*

Proof. First, we see that $g^i \equiv 1 \pmod{g-1}$, which can be proved by induction. By Lemma 3, $\mathsf{ord}(g) = \gamma$. Pigeon hole principle implies that the elements of $\{g^i \pmod{2d} \mid 0 \leq i < \gamma\}$ are precisely of the form $g + j(2d/\gamma)$ for $0 \leq j < \gamma$. Therefore, the map $g^i \pmod{2d} \mapsto g^i + g - 1 \pmod{2d}$ is a bijection. □

The next lemma is central to the proof of Lemma 1.

Lemma 5. *Let $p(X) = \sum_{j=0}^{d-1} c_j X^j$. Let $\gamma < d$ be a power of two integer and $g = 2d/\gamma + 1$. Then,*

$$\pi_\gamma(p) := \sum_{i=0}^{\gamma-1} \tau_g^i(p) \implies \pi_\gamma(p) = \gamma \sum_{\gamma | i} c_i X^i.$$

Proof. We have $\tau_g^i(p) = \sum_{j=0}^{d-1} c_j X^{jg^i}$ and so

$$\pi_\gamma(p) = \sum_{i=0}^{\gamma-1} \sum_{j=0}^{d-1} c_j X^{jg^i} = \sum_{j=0}^{d-1} c_j \Big(\sum_{i=0}^{\gamma-1} X^{jg^i} \Big).$$

Thus, it suffices to show that $\sum_{i=0}^{\gamma-1} X^{jg^i} = \gamma \cdot X^j$ if $\gamma \mid j$ and 0 otherwise.

Suppose that $\gamma \mid j$. It suffices to show that $X^{jg} = X^j$, which would imply that $X^{jg^i} = X^j$ for all $0 \leq i < \gamma$. Now, since $g = 2d/\gamma + 1$, $jg = 2dj/\gamma + j$. But since $\gamma \mid j$, it follows that $2dj/\gamma$ is a multiple of $2d$, which implies that $X^{jg} = X^{2dj/\gamma + j} = X^j$. Therefore, we have $\sum_{i=0}^{\gamma-1} X^{jg^i} = \gamma \cdot X^j$. On the other hand, suppose that $\gamma \nmid j$. We want to show that $\sum_{i=0}^{\gamma-1} X^{jg^i} = 0$. Let $\omega = X^{j(g-1)}$. Then,

$$\omega \sum_{i=0}^{\gamma-1} X^{jg^i} = \sum_{i=0}^{\gamma-1} X^{j(g^i+g-1)}.$$

By Lemma 4, the map $g^i \pmod{2d} \mapsto g^i + g - 1 \pmod{2d}$ is a bijection, which implies that

$$\sum_{i=0}^{\gamma-1} X^{j(g^i+g-1)} = \sum_{i=0}^{\gamma-1} X^{jg^i}.$$

This shows that $(\omega - 1) \sum_{i=0}^{\gamma-1} X^{jg^i} = 0$. It remains to show that $\omega \neq 1$, which would prove that $\sum_{i=0}^{\gamma-1} X^{jg^i} = 0$ because $\mathbb{Z}[X]/(X^d + 1)$ is an integral domain. But $\omega = X^{j(g-1)} = X^{2dj/\gamma} = 1$ if and only if $2dj/\gamma$ is a multiple of $2d$ if and only if $\gamma \mid j$. But since $\gamma \nmid j$, it follows that $\omega \neq 1$, completing the proof. □

Finally, we prove the main lemma using the above.

Proof of Lemma 1. Applying Lemma 5 with $\gamma = d/2$ and $g = 5$ yields $\pi_{d/2}(p) = (d/2)\left(c_0 + c_{d/2} X^{d/2}\right)$. Now

$$\tau_h \circ \pi_{d/2}(p) = \frac{d}{2}\left(c_0 + c_{d/2} \cdot \tau_h(X^{d/2})\right) = \frac{d}{2}\left(c_0 - c_{d/2} X^{d/2}\right)$$

But $\mathrm{Tr} = \pi_{d/2} + \tau_h \circ \pi_{d/2}$ implying $\mathrm{Tr}(p) = d \cdot c_0$. □

Appendix C.
Meta-Review

The following meta-review was prepared by the program committee for the 2026 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

C.1. Summary

This paper presents a new single-server private information retrieval (PIR) scheme that achieves high throughput and low query communication costs without requiring any offline communication, operating in the silent preprocessing model. At the core of this improvement is a novel ring-packing algorithm that actively leverages the common reference string (CRS) model to reduce the number of required rotation keys while also improving computational efficiency.

C.2. Scientific Contributions

- Creates a New Tool to Enable Future Science
- Provides a Valuable Step Forward in an Established Field

C.3. Reasons for Acceptance

1) The paper proposes a new PIR protocol which achieves great throughput and communication. These results are confirmed via experimental evaluations.
2) The paper introduces a novel ring-switching technique which achieves a 5x reduction in key size. This technique may be of broader interest in the scientific community.

C.4. Noteworthy Concerns

1) Many of the reviewers felt that the paper needs improvement in the presentation of the paper and its results. For instance, the paper would benefit from more intuition on the mathematical underpinnings of the construction and its proof. One reviewer suggested adding a diagram which demonstrates the transportation pipeline or including a toy example to help with understanding.
2) The paper is not comprehensive in its comparison with prior related work. In particular, while the paper cites various prior works, it only chooses a subset of these in its experimental comparisons and does not explain why this subset was selected. Furthermore, the paper is missing a better discussion of the trade-offs of using this work versus other prior related work in different settings. Lastly, the authors do not include an asymptotic comparison of their construction with prior related work.